Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

First step I performed was to import all the required versions of libraries which would help me with analysis, do computations and help us with visualising data. Numpy and Pandas were imported for performing computations and data analysis and manipulation while matplotlib and seaborn were imported for data visualization. Warnings library was imported to ignore all the unnecessary warnings that might occur while working with data in notebook.

Then, I tried to read and display info of only 1 file i.e., 2023-1.parquet file to understand how many columns, rows and memory of data is present in the file. This is to understand the size, structure and type of data I am dealing with.

After that I sampled data from all of the files by taking 5% of trips in each hour and each day of each month such that we have data for all months, for all days of weeks, and for each hour for our analysis. I did this by creating a list of all the .parquet files I had for each month and looping through each file one by one. Within each file, loop was set to go through each day and each hour and pickup 5% of data randomly and append it to my sampled_data file.

After sampling 5% of data from each file, total number of entries were coming around 18 – 19 lakhs, so sampled it again to have maximum entries of 300,000 in the sample file. After sampling, I viewed the info of the sampled_data file wherein I can see all the columns, their data types, total number of non-null values etc.

2. Data Cleaning

After creating sampled_data file, I loaded the newly created sample data into the dataframe and gave df.head() to check first 5 rows of the data for each column in the dataframe, what kind of data is present, what format the values are present in.

2.1. Fixing Columns

2.1.1. Fix the index

After loading data and viewing the top 5 rows of data and info of sampled data, I realized we have pickup_date and pickup_time column which is not required since we already have tpep_pickup_datetime which we can use to get this data. So, I dropped these 2 columns from the dataframe. I then reset the index to have only 1 index field and in sequence in the dataframe.

2.1.2. Combine the two airport_fee columns

Then there were two different airport_fee columns which seemed to have same kind of data, so I combined both the columns by populating all the values in airport_fee column for each row to Airport_fee column. After viewing the resultant values in the columns, I dropped airport_fee column so that we have only 1 column for airport fee.

I also checked if there are negative values in the monetary columns by adding all the monetary columns to a list and then filtering the data from dataframe for those columns if they have any value less than 0.

I put all the monetary columns in loop and got the count of all the rows that have negative value in any of those columns.

There were few rows where I can see that all the charges like improvement surcharge, mta_tax, extra etc. are in negative while fare_amount is 0 for them. This can be caused because of cancelled trips or trips that didn't happen. So, I dropped such rows from the the sampled data/dataframe.

I also analyzed RatecodeID for the negative fare amounts, but I observed that most of those rows have RatecodeID as 1.0 or 2.0. I checked all unique values for RatecodeID column and reached to an understanding that most of the rows with negative monetary values have RatecodeID as 1.0 which is for standard rate and since most of the normal trips have standard rate only so I did not change anything to this column.

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

To find the proportion of missing values in each column, I executed 'missing_value_mean = df.isnull().mean()'. This command would get total

number of rows that has missing values, and calculate its mean so that we get the proportion of null/missing values in each column. Then I printed the missing_value_mean. Since, the proportion for few columns with missing values was very less so I was getting 0.0 even if there is missing values for those columns. So, I added this condition, 'missing_value_mean.map(lambda x: f"{x:.6f}")' to the missing_value_mean to get the proportion upto 6 decimal places and then I could see few columns were having values missing like passenger_count, store_and_fwd_flag, RatecodeID, congestion_surcharge, Airport_fee.

2.2.2. Handling missing values in passenger_count

To handle missing values in passenger_count, first I filtered the rows that have null values in passenger_count column. Then, I displayed few rows to understand what null values does it has. I could see that those rows are displaying 'NaN' for passenger_count, so I ran the code to get all statistics related to passenger_count column like mean, median, count of all unique values in the column. After looking at the statistics, I found that mean is 1.36, median is 1 and the total number of rows with passenger_count as 1 is minimum more than 4 times of what other passenger_count has, so I decided to impute 'NaN' values in passenger_count column with '1'. Then, I ran code to calculate proportion again to make sure that null_value proportion in passenger_count column has become 0

2.2.3. Handle missing values in RatecodeID

To fix missing values in RatecodeID, again I got all proportions and then calculated all statistics for RatecodeID column like mean, median, count of all unique values in the column. After calculating statistics, I found that mean is 1.6, median is 1 and RatecodeID as 1 has most number of entries, so for RatecodeID also, we can replace null values with 1. I also noticed that RatecodeID has '99' which is not mentioned in data definition, but upon checking the shape of RatecodeID with value '99' the number of rows seemed to be on lower side only, so replaced all rows with '99' as RatecodeID to '1'

2.2.4. Impute NaN in congestion_surcharge

To handle null values in congestion_surcharge, again I got all proportions, then displayed all the rows with congestion_surcharge as null. Since, this column can have null values because there was no congestion_surcharge at all, so I replaced all null values with '0'

Similarly, for airport_fee, store_and_fwd_flag also, I checked the proportion and then handled the missing values. In airport_fee just like congestion_surcharge, null seems to be because of no airport fee charge for that trip, so replaced null values with '0' while in store_and_fwd_flag, number of rows with null values were more, so instead of imputing it with any other already existing value, I renamed it for now to 'U' for unknown

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

First, I described the data to get all numerical analysis of sample_data, then I observed that columns like passenger_count, trip_distance, fare_amount, extra, mta_tax, tip_amount, tolls_amount, total_amount, airport_fee has huge gap in their 75th percentile and max values. so they could be required to be handled for any outlier. I started checking values for each column for possible outliers and found there are some columns where outliers can be handled like passenger_count having value more than 6 is very less, trips with 0 distance and fare but different pickup and dropoff zones seemed like invalid data. Similarly, trips with 0 distance but fare more than 300 also seemed invalid as even if a very short trip happened, fare can't be that much. So, I dropped all such rows mentioned above. I also dropped rows with distance more than 250, tip amount more than 20, tolls amount more than 10 as it was affecting overall numerical analysis and was acting like outlier. Then I dropped rows with payment mode as 0 and total_amount as 0 as these values seemed invalid for taxi trips.

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

Categorise the varaibles into Numerical or Categorical.

VendorID: Categorical

tpep_pickup_datetime: Categorical

tpep_dropoff_datetime: Categorical

passenger_count: Numerical

trip_distance: Numerical

RatecodeID: Categorical

PULocationID: Categorical

DOLocationID: Categorical

payment_type: Categorical

pickup_hour: Numerical

trip_duration: Numerical

The following monetary parameters belong in the same category, is it categorical or numerical?

These monetary parameters are all **Numerical**

fare_amount

extra

mta_tax

tip_amount

tolls_amount

improvement_surcharge

total_amount

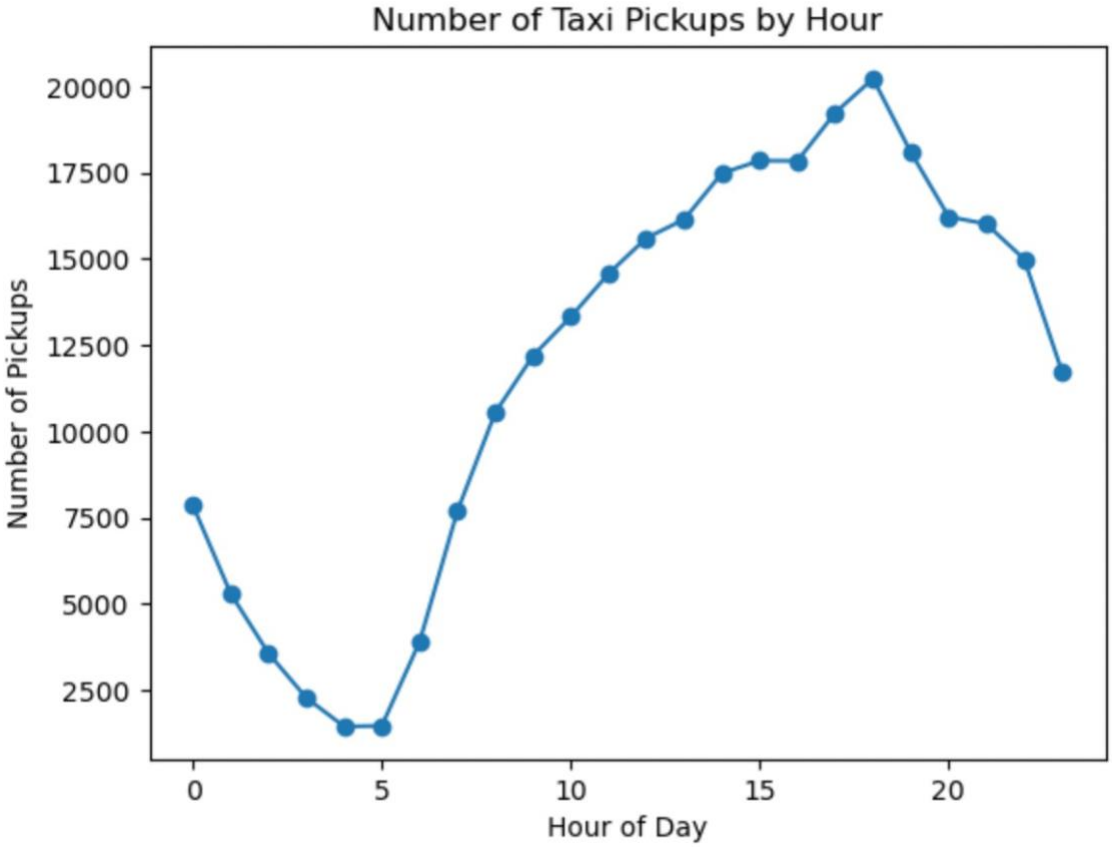
congestion_surcharge

airport_fee

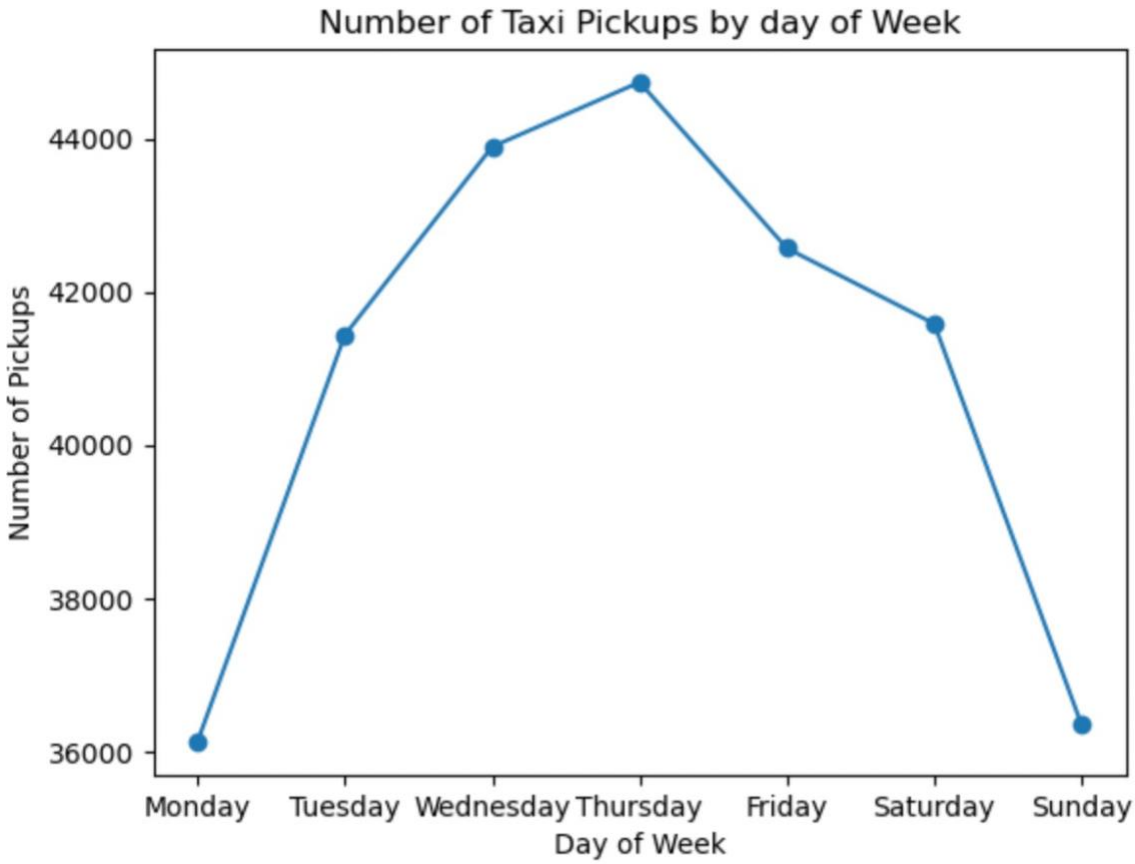
3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

To analyze distribution of taxi pickups by hours, days of the week, and months, first I extracted pickup hour from tpep_pickup_datetime and added it to the dataframe. Then, I calculated total number of taxi pickups in each hour and filtered hour and count of pickups in new variable called hourly_count. Then I plotted the data in hourly_count as shown below. From looking at the plot, I can say that number of taxi pickups peak during the evening time and it is lowest during early morning time. This could be because during early morning, people are at home, they start booking taxis when starting for work and so the plot shows graph going up and it peaks during evening as people might be booking cab to come

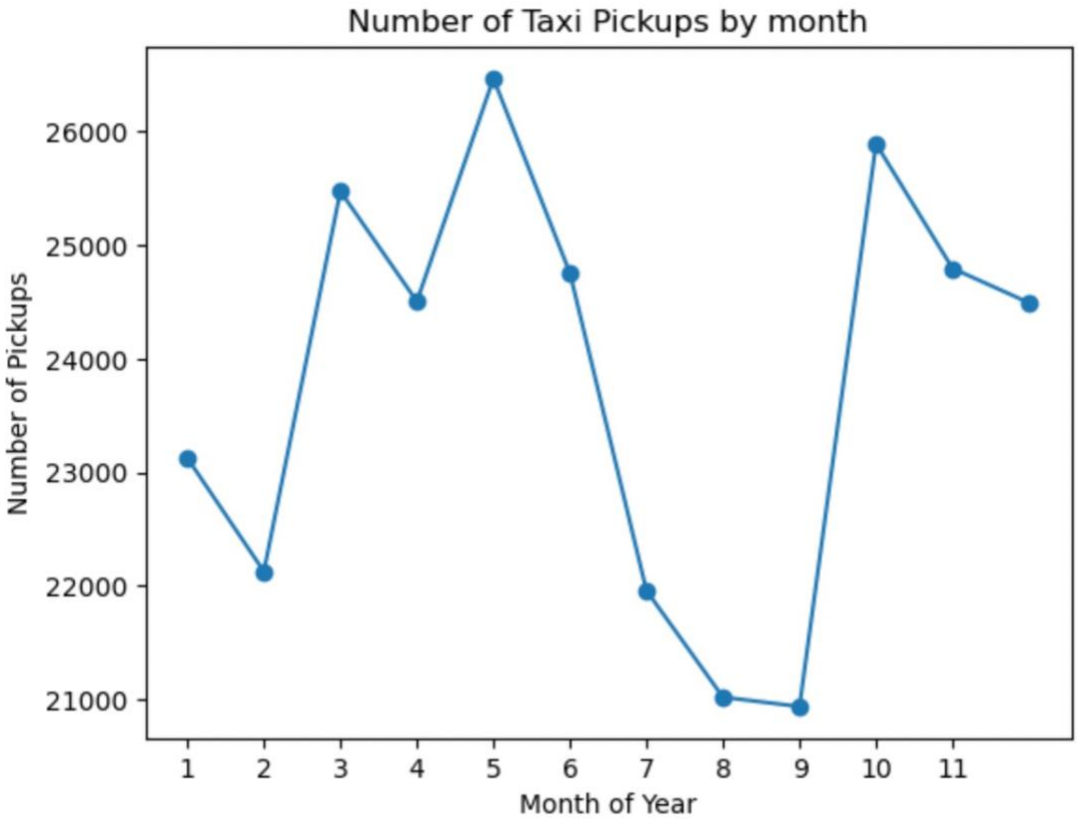
back from office, from commercial districts or from entertainment hubs. There is a decline in pickups during night as very less people travel during night time because that is not a office time or shopping, entertainment hubs would be open where people can go to



Similarly, for days of week, I extracted day of the week from tpep_pickup_datetime, map the day number with their day names, and store the data in weekday_pickup_count variable. I also reindexed the variable to have all days in sequence starting from Monday. Then I plotted the graph. As can be seen from graph, the peak is in the mid of the week as most of the people commute to and from office and bookings near weekends are lesser around weekends as people might prefer doing work from home



To analyze monthly trends in pickups, I extracted month from tpep_pickup_datetime field, and calculated total number of pickups in each month and store it in monthly_count variable. Then I plotted the data in monthly_count as in below graph. From the plot, I could say that, the pickups are more around months like March, May and October while it is lesser in months like February, August and September. What I understood is it can be because people prefer travelling more during spring, summer or autumn season and tourism is also more as it is peak season for travel while during cooler seasons around February, travel could be less because it is very cold and also tourism is less because of off season.



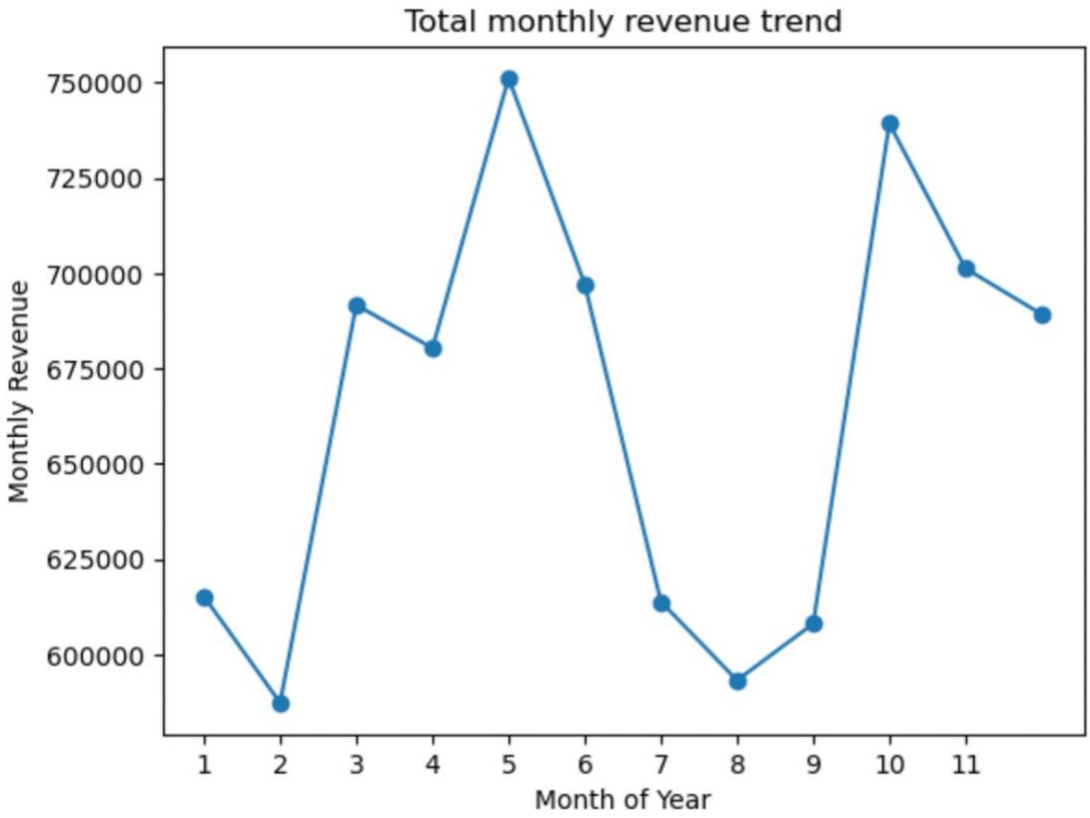
3.1.3. Filter out the zero/negative values in fares, distance and tips

First to check if these parameters have negative values, I stored all these columns in 1 list, then loop through the list to count total number of rows with zeroes and negative values in these columns. I found that no column has negative value since negative values have already been taken care of in previous steps. But, tip_amount has lots of rows with zeroes that could be caused by passengers not tipping after trips. Then there are zeroes in fare_amount and trip_distance as well. As per my analysis, trip_distance can also be zero if trips are very short. However, filtering rows with fare_amount zero can help in cleaning data because these trips are either cancelled or the data is invalid. I created a new dataframe 'df_non_zero' with all these columns not having any zeroes as values.

3.1.4. Analyse the monthly revenue trends

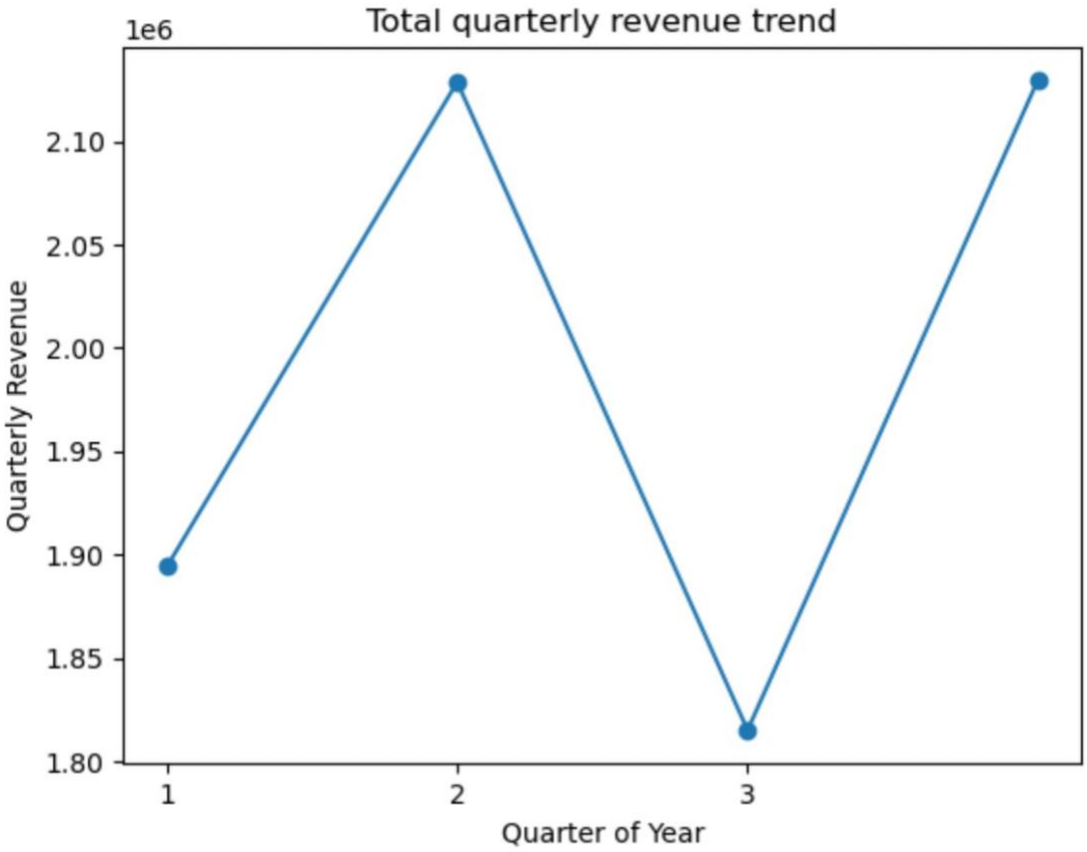
To analyze monthly trend, I used already extracted pickup month in previous steps and grouped the dataframe by pickup month and calculated sum of total_amount available in each row to get the total revenue for that month and saved the data in monthly_revenue variable. I

then plotted the data in montlhy_revenue as shown below. By looking at the plot, I can say that the monthly revenue was more around month of March, peaked around May and October, while it was less during February, August and September. This is expected trend in revenue as we see in previous step that pickup trend was also same for each month. So, if pickups are less, the trips will be less and so revenue in result will also be less.



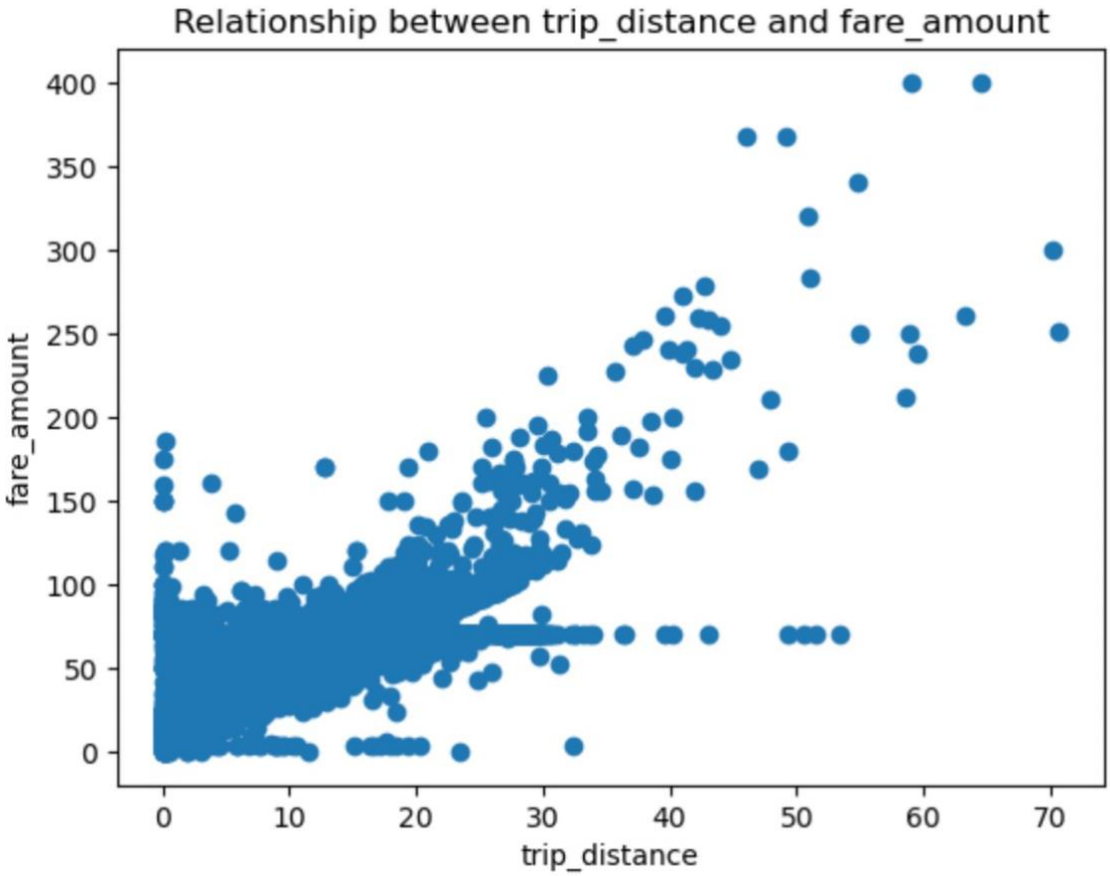
3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

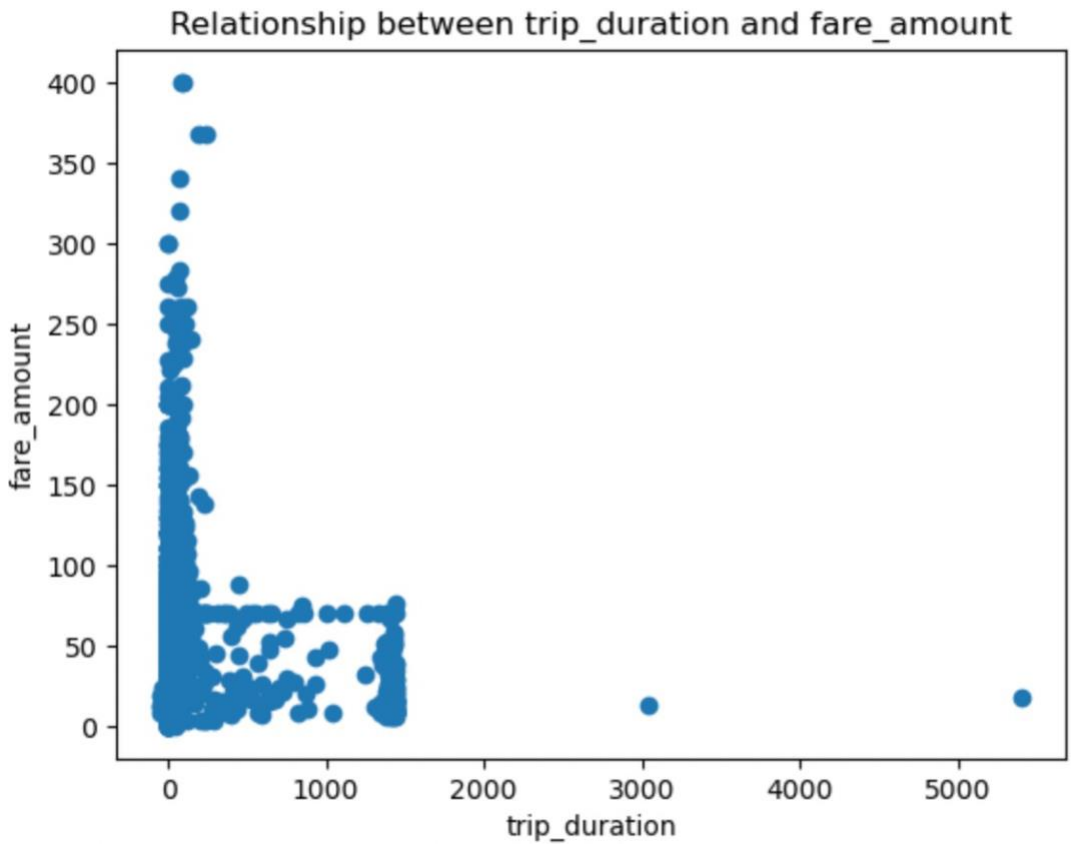
To calculate proportion of each quarter revenue, first I extracted quarter from tpep_pickup_datetime field and then grouped the dataframe by pickup quarter, calculated the sum of total_amounts available in each row to get total revenue for every quarter and stored the data in quarterly_revenue variable. After that, I plotted the graph for data in quarterly_revenue variable. From the graph, I can observe that revenue seems to be higher in second and fourth quarter and lesser in first and 3rd quarter. This could be caused because people prefer to travel more during summer and holiday season, so travel is more during that time while it is lower in other quarters because travel and tourism both is less.



3.1.6. Analyse and visualise the relationship between distance and fare amount

To understand how trip fare is affected by distance, first I filtered out all the trips where trip_distance is 0. Then I plotted a scatter plot to understand the relationship between fare_amount and trip_distance as shown below. As we can see from the scatter plot, fare_amount seems to have positive correlation with trip_distance as there is an increase in fare_Amount with increase in trip_distance. I also calculated correlation between fare_amount and trip_distance and the result is 0.94 which supports the theory from the scatter plot. I also see that there is more density for lower fares and for higher fare amount, the density reduced. As per my analysis, this could be caused because for shorter trips, there is only base amount that is calculated for a fixed amount of distance. After those fixed distance, extra charges like incremental charges for each kilometer/mile starts to add. Similarly, we can see in the plot also, there are dots in straight line for distance value upto around 55 and fare amount '75' seems to be same for that distance. This might be because a particular vendor had set base amount for upto 55 miles/kms as 75 only.

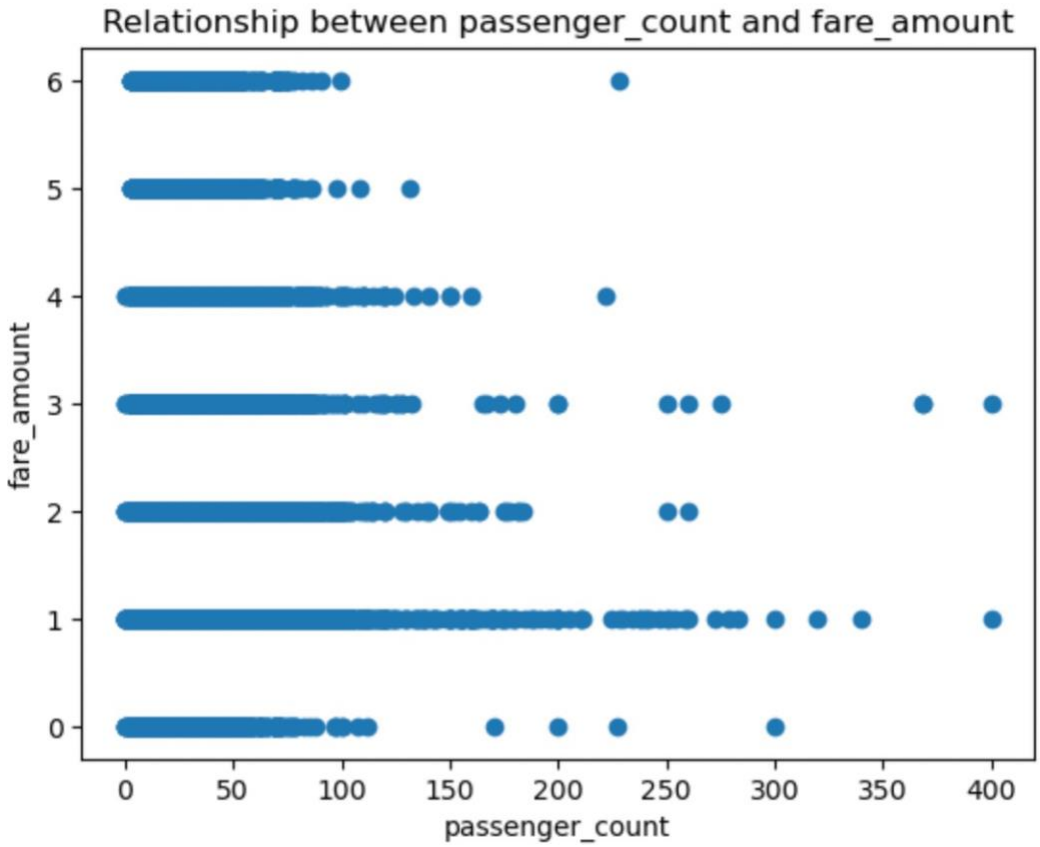

Relationship between trip_distance and fare_amount
fare_amount
trip_distance
400
350
300
250
200
150
100
50
0
0
10
20
30
40
50
60
70

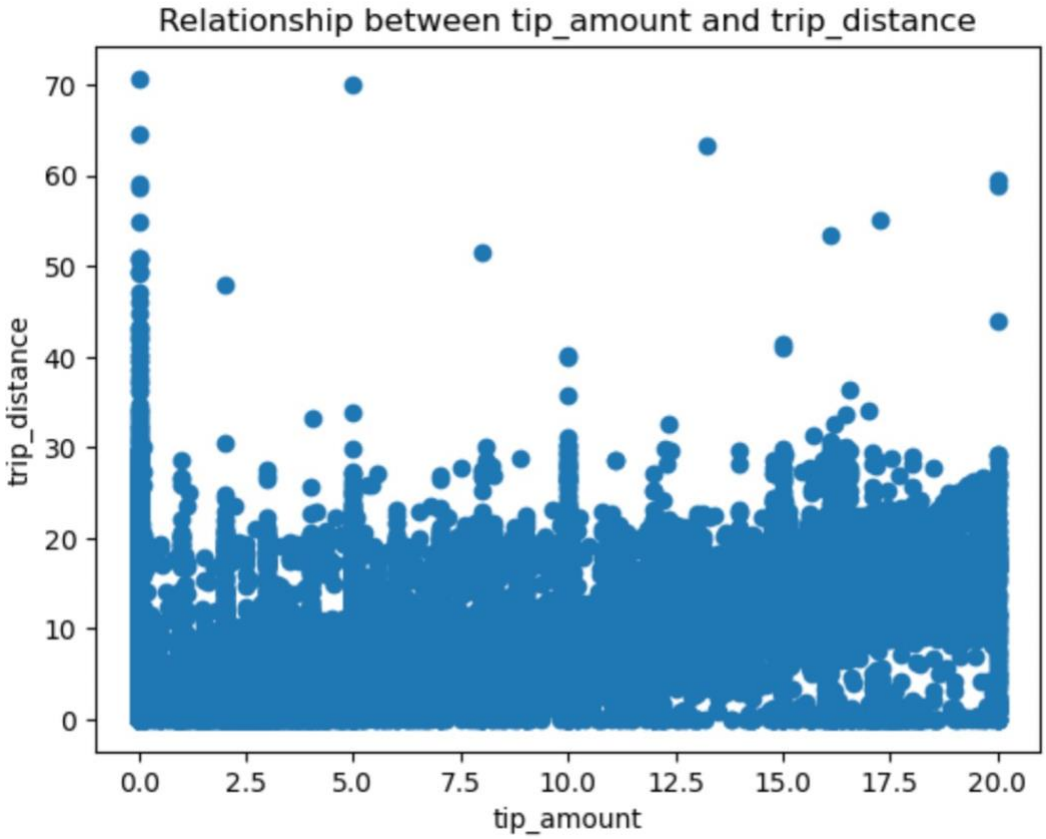


3.1.7. Analyse the relationship between fare/tips and trips/passengers

Relationship between fare amount and number of passengers can be seen from below scatter plot. The correlation between passenger count and fare amount came out to be 0.0427 which is near to 0 suggesting that there is no correlation between both the columns. That means, fare amount doesn't really depend on how many passengers are travelling.

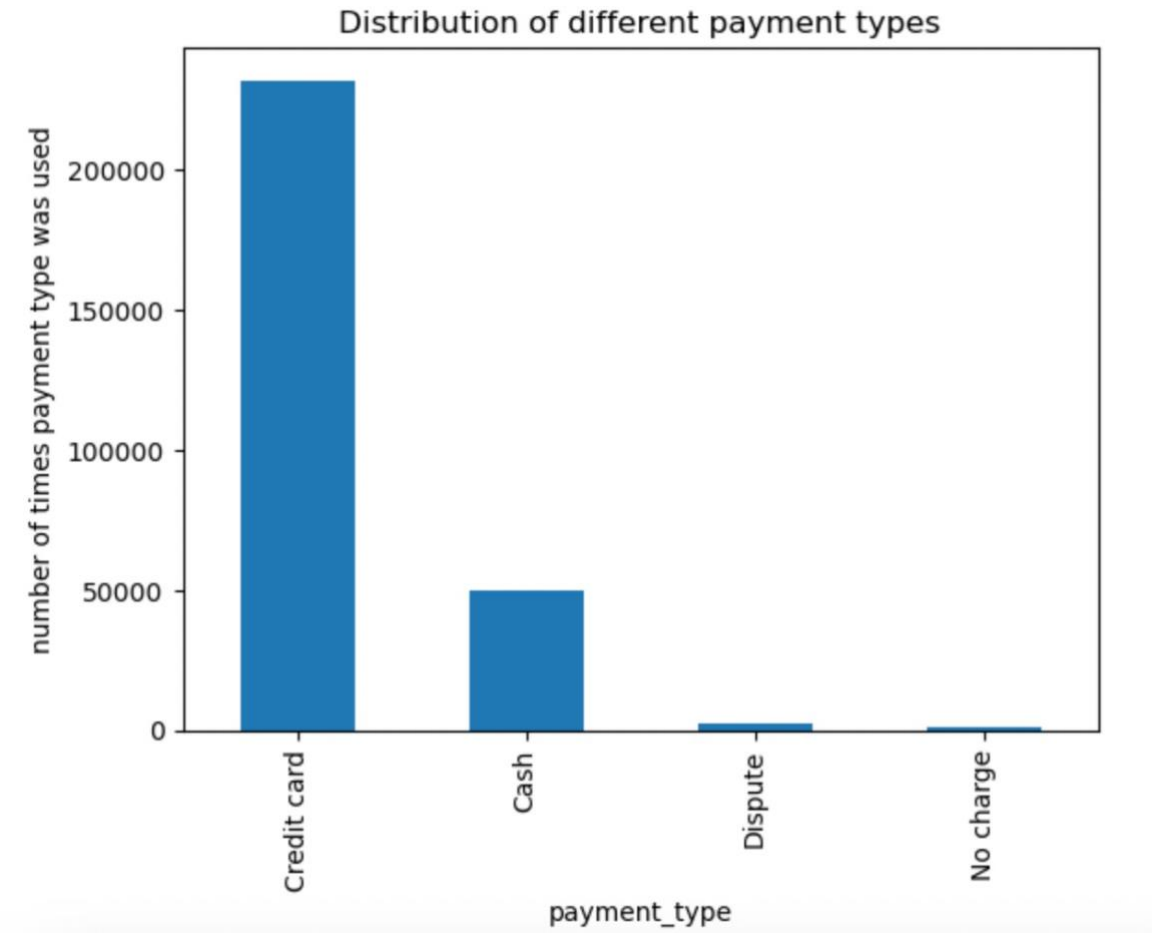
Then, there is another scatter plot showing relationship between tip amount and trip distance. From plot, we can see that there is some correlation between both the columns but they are not highly correlated which was confirmed by the correlation calculated between both the columns that came out to be 0.56 suggesting that to some extent, tip amount depends on the fare amount like if fare amount is more, tip amount will be more but entirely it is not true.


Relationship between passenger_count and fare_amount
fare_amount
passenger_count
0
1
2
3
4
5
6
0
50
100
150
200
250
300
350
400



3.1.8. Analyse the distribution of different payment types

Distribution of different payment types was calculated by taking all possible options in dictionary, mapping it to the payment_type values available in dataframe and then calculating total number of rows for each payment type. As we can see from the plot below, credit card seems to have most entries, followed by cash. This could be because in current digitalized world, people often prefer using credit cards over cash, as it is easy to carry plus credit cards have its own benefits.

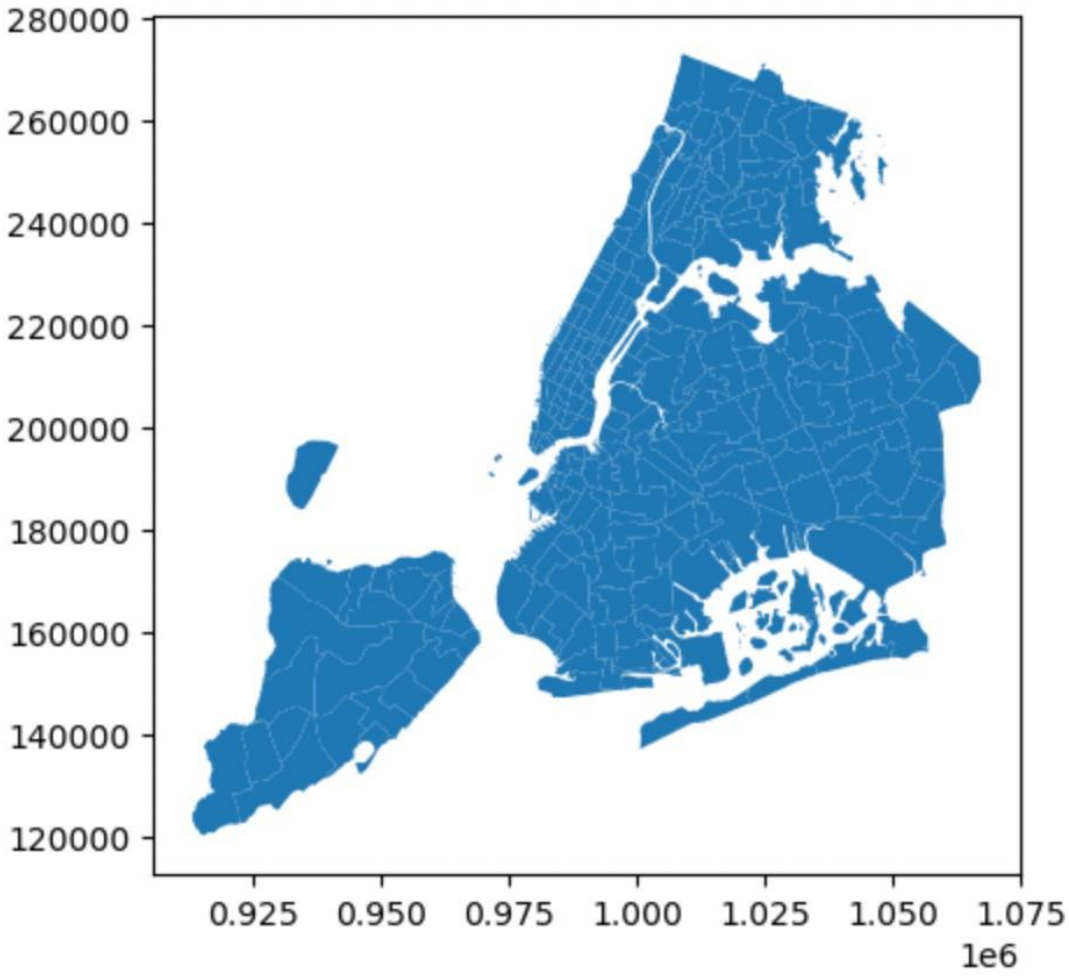


3.1.9. Load the taxi zones shapefile and display it

To load the taxi zones shapefile, I first installed geopandas library and imported geopandas library as it is required to work with shape file. Then, I read the shape file available for taxi trips in different zones and checked the type of data available by using head() function to view first 5 rows in the table. Then I also plotted the same by running zones.plot() code

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry
0	1	0.116357	0.000782	Newark Airport	1	EWR	POLYGON ((933100.918 192536.086, 933091.011 19.
1	2	0.433470	0.004866	Jamaica Bay	2	Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343.
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3	Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2.
3	4	0.043567	0.000112	Alphabet City	4	Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20.
4	5	0.092146	0.000498	Arden Heights	5	Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144.

<Axes: >



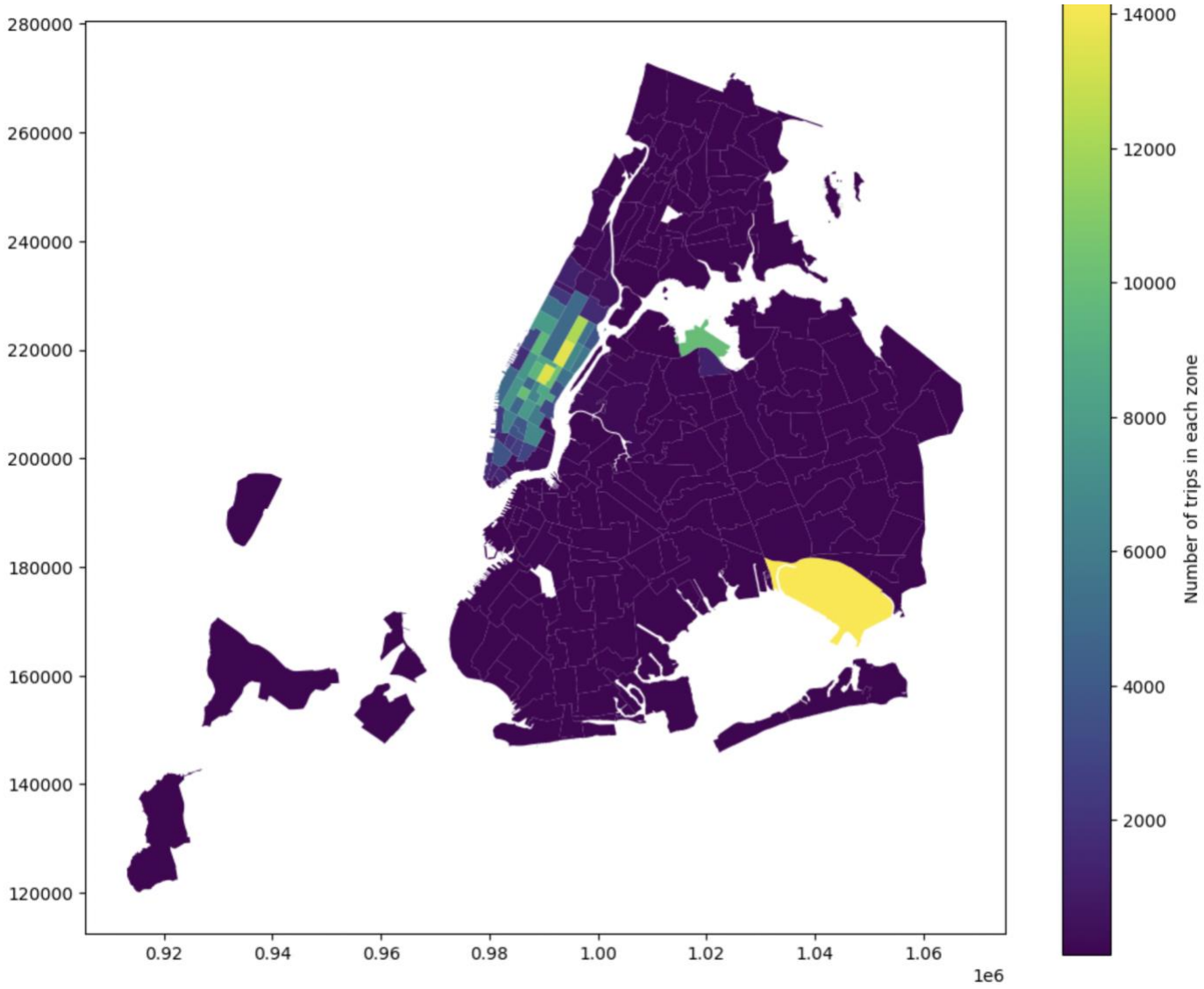
3.1.10. Merge the zone data with trips data

Now, to merge both the dataframes i.e., zone data with trips data, I used .merge() function to join both the tables by keeping trips data on left side, zones table on right side and left joining both the table on PULocationID and LocationID respectively so that all data from trips data comes in output and zones data whichever is available can be joined to trips data.

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry	PULocationID	num_
0	1	0.116357	0.000782	Newark Airport	1	EWR	POLYGON ((933100.918 192536.086, 933091.011 19...	1.0	
1	2	0.433470	0.004866	Jamaica Bay	2	Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343...	NaN	
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3	Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2...	3.0	
3	4	0.043567	0.000112	Alphabet City	4	Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20...	4.0	
4	5	0.092146	0.000498	Arden Heights	5	Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144...	NaN	

3.1.11. Find the number of trips for each zone/location ID

After merging both the tables on location IDs, I grouped the data by PULoactionID to find out total number of trips happened per location id and then plotted a color-coded map to visualize the density of number of trips in each zone as shown below



3.1.12. Add the number of trips for each zone to the zones dataframe

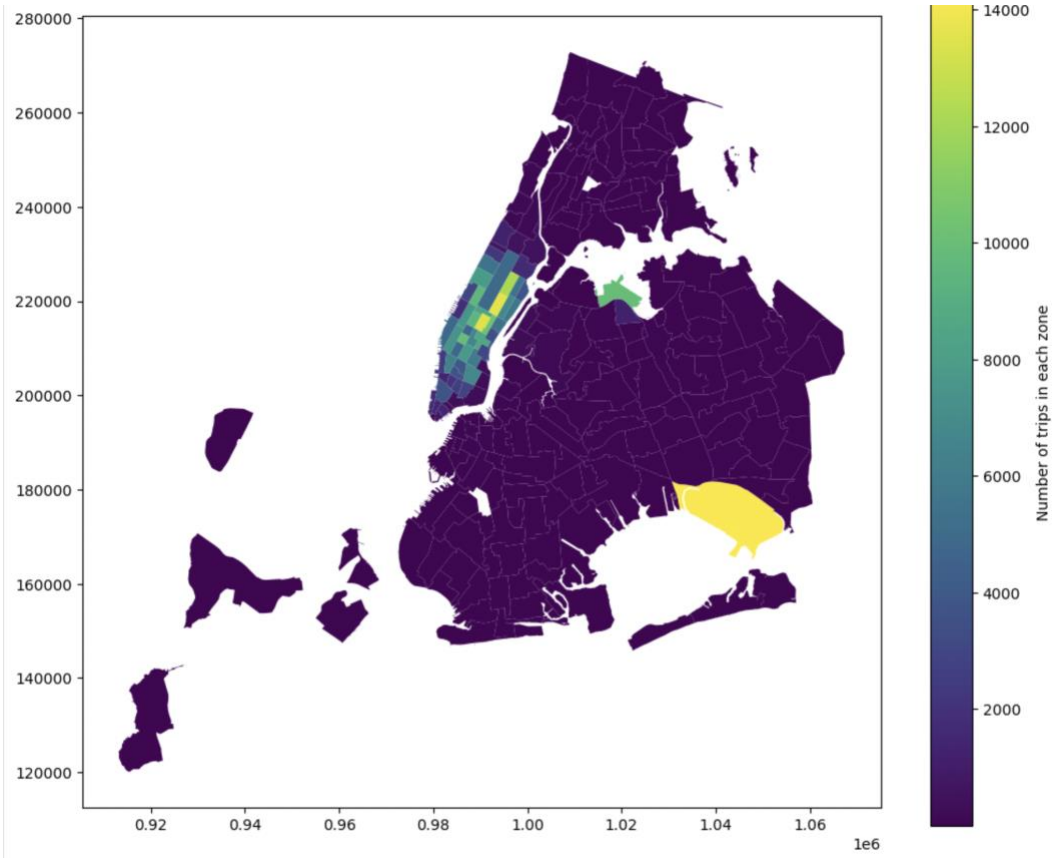
While grouping the data by PULocationID, I reset the index and added ‘num_of_trips’ column before calculating the number of trips happened in each loaction so that there is an additional column added to the merged table with total number of trips happened for that location id

Group data by location IDs to find the total number of trips per location ID

211]:
```
# Group data by location and calculate the number of trips
num_trip_grpby_location = df.groupby('PULocationID').size().reset_index(name = 'num_of_trips')

print(num_trip_grpby_location)
```

```
     PULocationID  num_of_trips
0               1            24
1               3             4
2               4           287
3               7           124
4               8             1
..            ...           ...
230           261          1414
231           262          3774
232           263          5445
233           264          2757
234           265           100

[235 rows x 2 columns]
```

3.1.13. Plot a map of the zones showing number of trips



3.1.14. Conclude with results

From above analysis on how number of trips vary for different zones, hour of day, day of the week, month, quarter and how fare_amount vary with distance, how it doesn't vary with number of passengers etc. I have come to conclusion that demand for taxis, the pickups and dropoffs are highest during commute to and from office i.e, when people are going to or coming back from office during the weekdays , demand peaking in mid week as people might be preferring to work from home around weekends and in the nights, demand is high in the zones with more entertainment

and commercial hubs during the weekends. I can also say that demand of taxis are higher during peak travel season as seen above during the months of May in summer and October in winters as those are time people go out in open as weather is pleasant and in winters for holidays, while other months, the traffic is lesser. I can also see that revenue trend matches with the number of trips across months which is expected as more number of trips will result into more revenue. I can also see that fare_amount varies with trip_distance and they are highly correlated but the scatter plot also can be seen constant around same fare for few distance suggesting that base amount remains same for few miles and has incremental charges after a fixed distance. Tip amount however doesn’t depend on fare amount much as it depends on person to person to give tip and remains almost constant even for high distance trips.

3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

to identify slow routes, I calculated trip duration from pickup and dropoff time and then grouped them by PULocationID, LocationID and pickup hour and taking aggregate of trip_distance and trip_duration. I then calculated speed from the mean trip distance and mean trip duration and then sorted out the grouped by table on speed in ascending order to find out the slowest trips

	PULocationID	DOLocationID	pu_hour	trip_distance	trip_duration	\
5974	50	239	1	1.770000	-12.838889	
12824	90	88	1	3.750000	-34.516667	
55594	238	263	1	1.490000	-22.158333	
43839	193	193	16	0.000000	0.408333	
23372	137	137	2	0.000000	0.233333	
...	...	...	...	...	...	
58012	248	248	22	16.700000	0.416667	
62569	265	265	8	23.366667	0.483333	
43923	197	197	2	2.500000	0.050000	
1374	23	23	10	20.300000	0.366667	
19582	130	130	13	19.400000	0.033333	

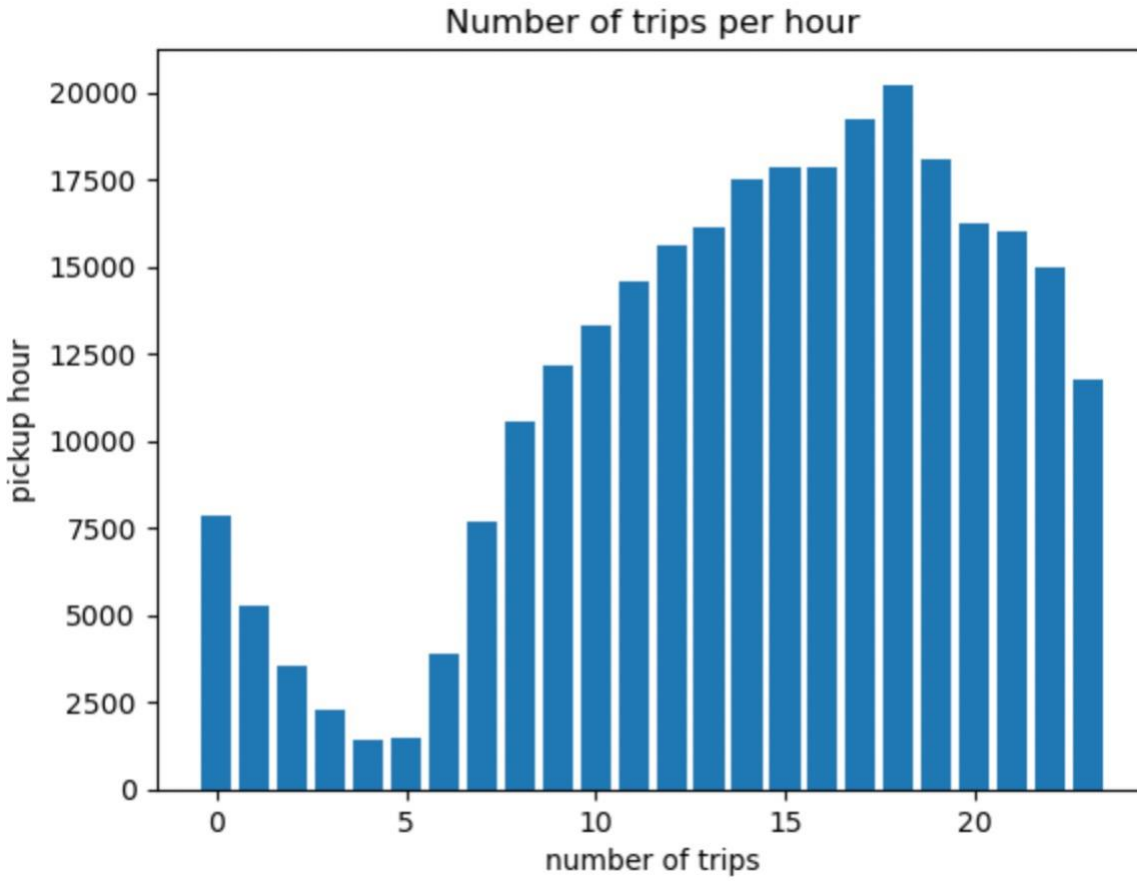
	speed
5974	-0.137862
12824	-0.108643
55594	-0.067243
43839	0.000000
23372	0.000000
...	...
58012	40.080000
62569	48.344828
43923	50.000000
1374	55.363636
19582	582.000000

[62585 rows x 6 columns]

This helped us in identifying what are all the routes that have high traffic and high demand which would let the vendors arrange/distribute the taxis in the high demand zones/routes accordingly and also let them identify the alternative routes for high traffic routes so that there is lesser traffic between same zones.

3.2.2. Calculate the hourly number of trips and identify the busy hours

After calculating speed for trips zone wise and then sorting the speed in ascending order, I calculated total number of trips in each pickup hour and then I plotted number of trips per hour to get the busiest hours as shown below. Since we took a sample of the data, the actual number is higher than shown here. From the plot, I can conclude that the most busiest hours of the day are between 08:00 and 23:00 hours. As already observed above, these are the hours when people are commuting to and from their work or going for shopping or to entertainment hub etc. and that is why number of trips are more between these hours.



3.2.3. Scale up the number of trips from above to find the actual number of trips

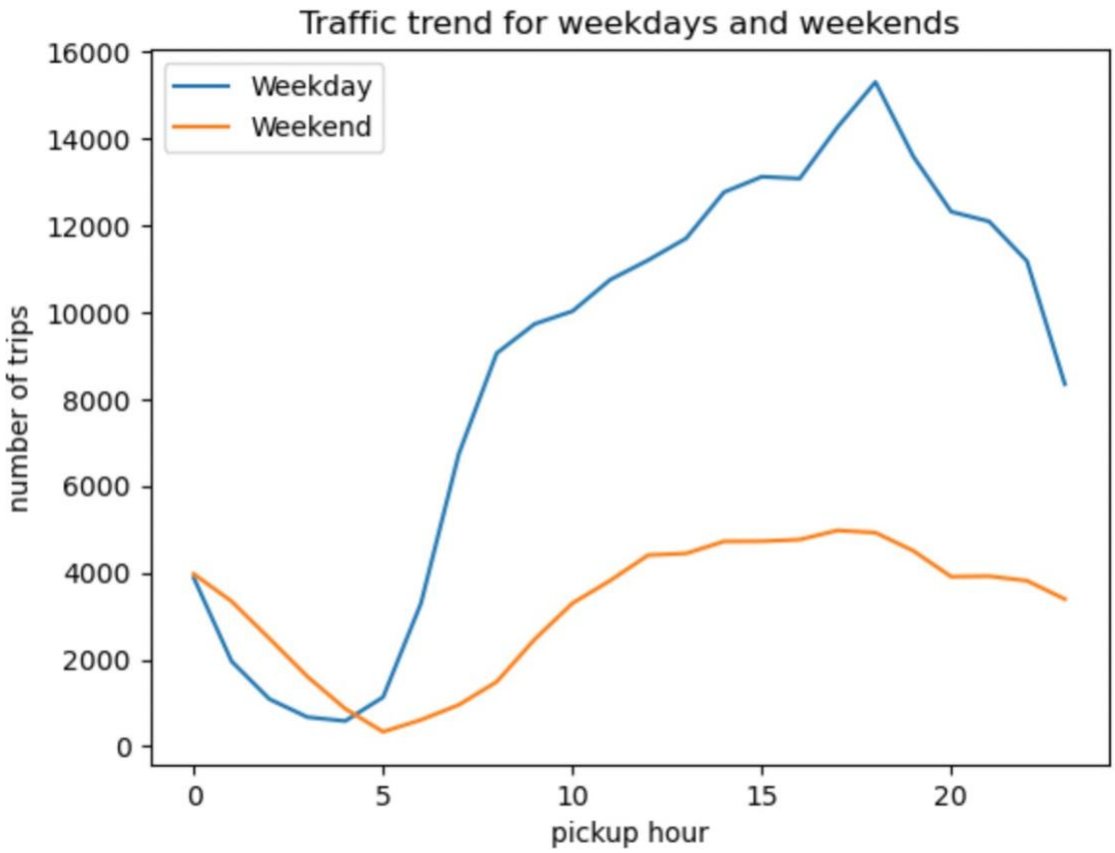
To scale up the number of trips to actual dataset, I divided the top 5 busiest trips count with the sampling function, '0.5' here to get the approximate number of trips for overall dataset as shown below

```
# Scale up the number of trips

# Fill in the value of your sampling fraction and use that to scale up the number
sample_fraction = 0.05

top_five_busiest_trips = trips_per_hour.sort_values(ascending = False).head()
#print(top_five_busiest_trips)

actual_trip_count_top_five_busiest = top_five_busiest_trips / sample_fraction
print(actual_trip_count_top_five_busiest)
```

```
pickup_hour
18    404460.0
17    384740.0
19    361780.0
15    356900.0
16    356800.0
Name: count, dtype: float64
```

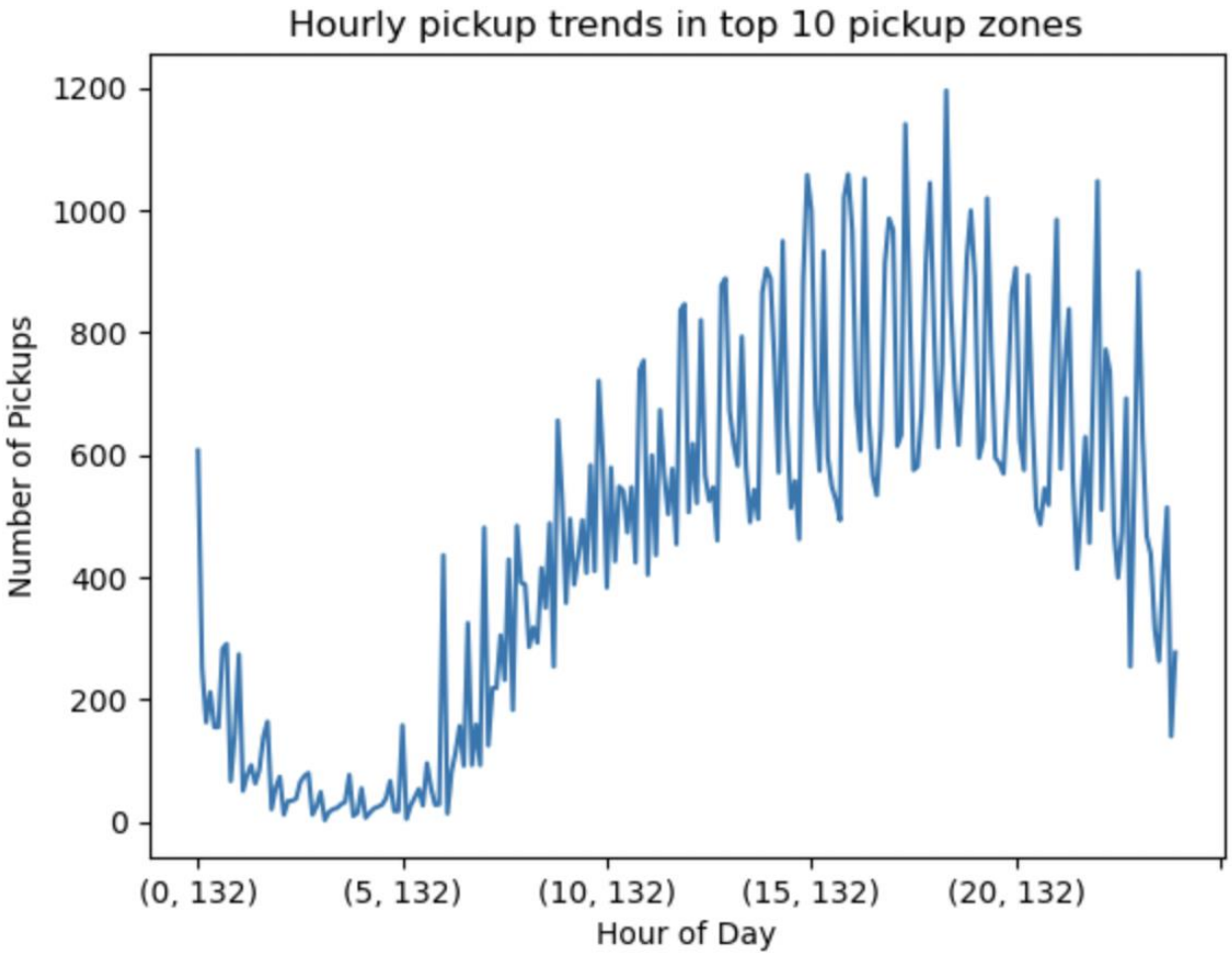
3.2.4. Compare hourly traffic on weekdays and weekends

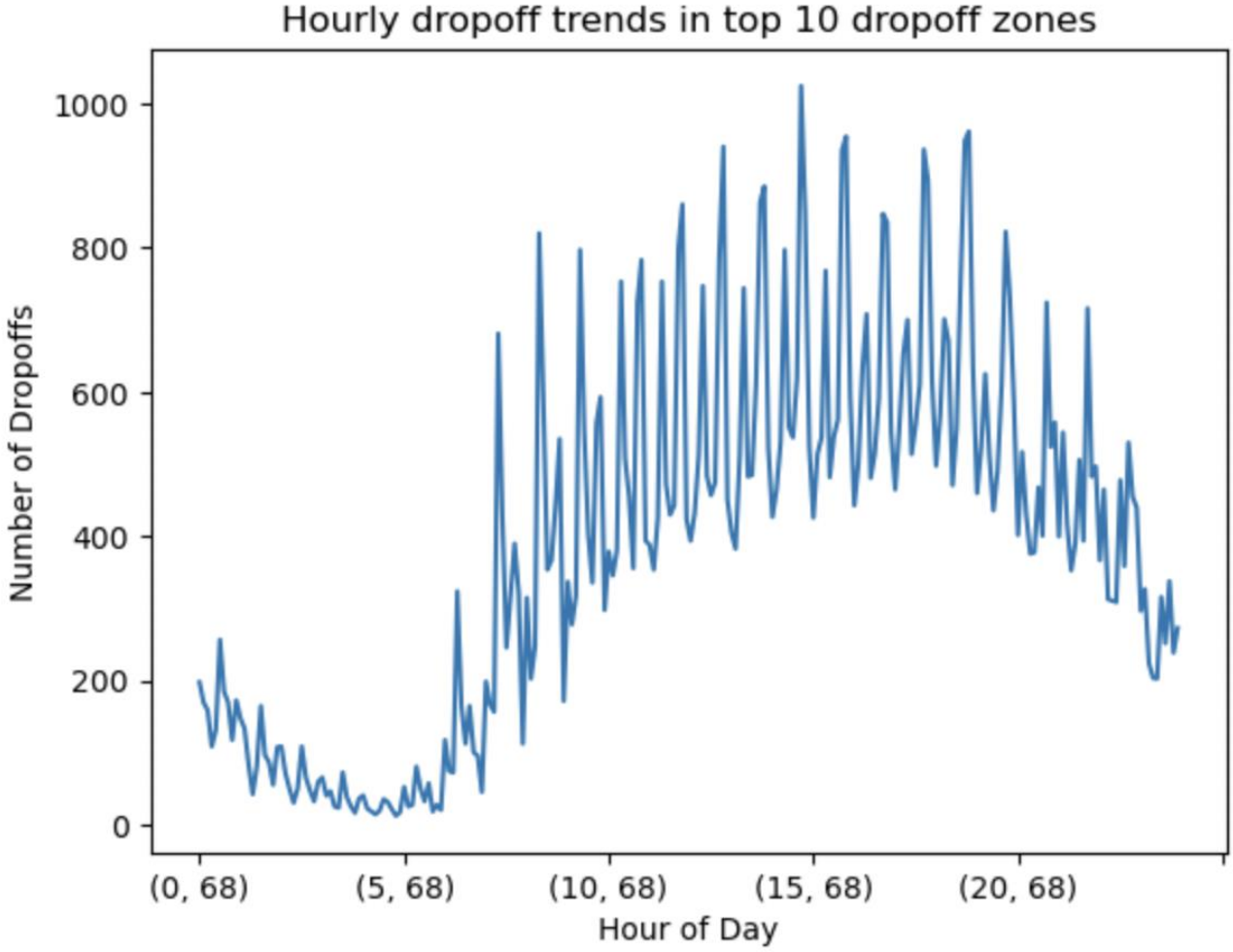
To compare the hourly traffic on weekdays and weekends, I created two different dataframes for both weekdays and weekends, and then plotted number of trips with pickup hour both weekdays and weekends as shown below. We can see that number of trips during weekdays are more than weekends which could be caused because people are going to and coming from office uses taxis for commuting. We also see that the traffic trend went down around end of the week days which could be because people prefer to do work from home near weekends as mentioned before. We can also see that the traffic is lower during early morning as less people book taxis to go anywhere during that time and traffic increases between 08:00 and 23:00 hours for both weekdays and weekends. From this plot, we can say that during early morning, the traffic is less and during day time, traffic is at peak. So, taxi distribution can be done in such a way that more taxis are made available during day time and on weekdays as that is when demand for taxis is highest.



3.2.5. Identify the top 10 zones with high hourly pickups and drops

I calculated the pickup and dropoff counts based on the PULocationID and then filtered out top 10 zones for both pickups and dropoffs based on their counts. Then, I grouped the data by hour to get the hourly trend for both top 10 pickup and dropoff zones. Finally, I plotted the data for number of pickups/dropoffs based on each hour for top 10 pickup and dropoff zones as shown in below plots. We can see that the hourly pickup and dropoff trends in top 10 zones matches to the overall traffic trend for each hour we plotted in above steps. These zones could be having more offices and other hubs like shopping malls or entertainment hubs and that's why the traffic trend in these zones matches with overall traffic trend.


Hourly pickup trends in top 10 pickup zones
Number of Pickups
1200
1000
800
600
400
200
0
(0, 132)
(5, 132)
(10, 132)
(15, 132)
(20, 132)
Hour of Day


Hourly dropoff trends in top 10 dropoff zones
Number of Dropoffs
1000
800
600
400
200
0
(0, 68)
(5, 68)
(10, 68)
(15, 68)
(20, 68)
Hour of Day

3.2.6. Find the ratio of pickups and dropoffs in each zone

To find out the ratio of pickup and dropoffs, first I created a new dataframe with pickup and dropoff count for each zone. Then I divided count of pickups with count of dropoffs for each zone as shown below. I then sorted out the dataframe on ratio in descending order and then filtered out top 10 and bottom 10 ratios to find out the zones where pickup:dropoff ratios are highest and lowest.

```
# Find the top 10 and bottom 10 pickup/dropoff ratios
ratio_df = pd.DataFrame({'pickups': pickup_counts,'dropoffs':dropoff_counts})
ratio_df['pickup_dropoff_ratio'] = ratio_df['pickups'] / ratio_df['dropoffs']

top_10_ratio = ratio_df.sort_values('pickup_dropoff_ratio', ascending = False).hea
bottom_10_ratio = ratio_df.sort_values('pickup_dropoff_ratio', ascending = True).

print('Top 10 pickup/dropoff ratio',top_10_ratio)
print('Bottom 10 pickup/dropoff ratio',bottom_10_ratio)
```

```
Top 10 pickup/dropoff ratio      pickups  dropoffs  pickup_dropoff_ratio
70    1260.0       147             8.571429
132  14194.0      3076             4.614434
138   9946.0      3512             2.832005
186   9978.0      6404             1.558089
43    4911.0      3533             1.390037
249   6395.0      4849             1.318829
114   3706.0      2839             1.305389
162  10400.0      8260             1.259080
100   4756.0      3966             1.199193
161  13508.0     11325             1.192759
Bottom 10 pickup/dropoff ratio      pickups  dropoffs  pickup_dropoff_ratio
67       1.0        40             0.025000
198      5.0       142             0.035211
92      10.0       229             0.043668
243     21.0       467             0.044968
37      13.0       273             0.047619
217      6.0       119             0.050420
112     33.0       641             0.051482
171      3.0        52             0.057692
49      24.0       395             0.060759
175      2.0        32             0.062500
```

3.2.7. Identify the top zones with high traffic during night hours

To get the top zones where traffic is high during night hours i.e., 11pm to 5 am, I first created a new dataframe by filtering out the rows where the pickup/dropoff was not between 11pm and 5am. Then, I used value_counts() function on PULocationID and DOLocationID to get the top 10 zones where the pickup and dropoff is highest between 11pm and 5am. Finally printed the top 10 zones with highest pickup and dropoff during night hours.

```
Top 10 pickup locations during night hours is PULocationID
79     2478
132    2116
249    1951
48     1611
148    1543
114    1347
230    1248
186    1067
138     960
68      945
Name: count, dtype: int64
Top 10 dropoff locations during night hours is DOLocationID
79     1292
48     1068
170     989
68      912
107     861
249     827
263     794
141     793
230     729
236     710
Name: count, dtype: int64
```

3.2.8. Find the revenue share for nighttime and daytime hours

To find the revenue share for nighttime and daytime hours, first I filtered out the rows with trips during night hours to get a dataframe with all daytime trips similarly as I did for nighttime trips in previous step. Then, I summed up the total_amount for all day trips as well as night trips filtered above. To get the revenue share and percentage, I calculated the fraction of daytime amount sum and nighttime amount sum to that of whole day amount sum and then calculating the percentage to find out the daytime revenue share and night time revenue share respectively.

```
Night time revenue and percent share is  950734.01  &  10.660700032369718
Day time revenue and percent share is  7967385.879999999  &  89.33929996763028
```

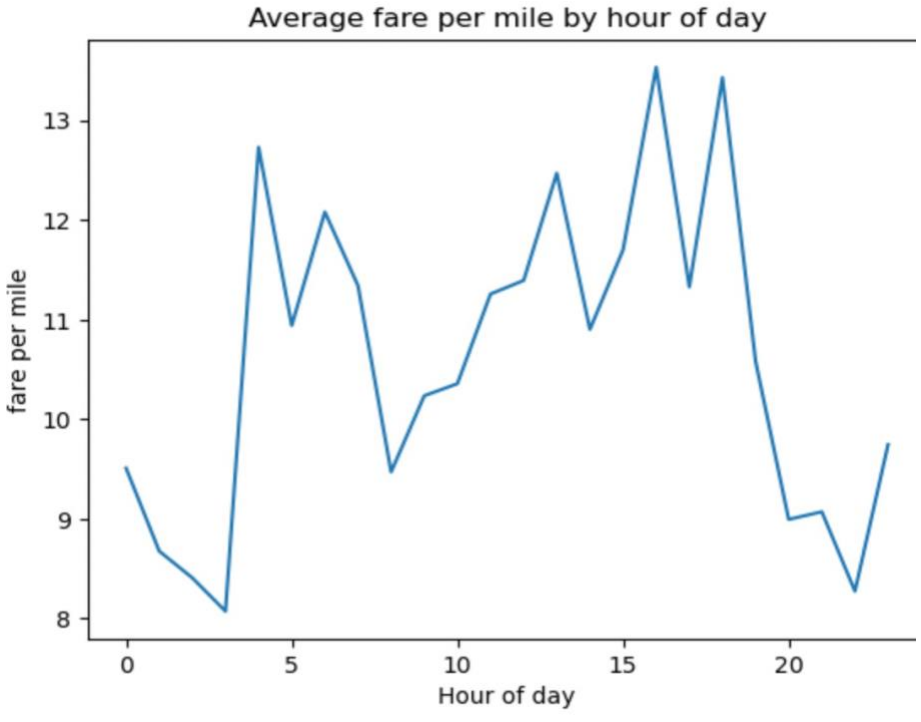
3.2.9. For the different passenger counts, find the average fare per mile per passenger

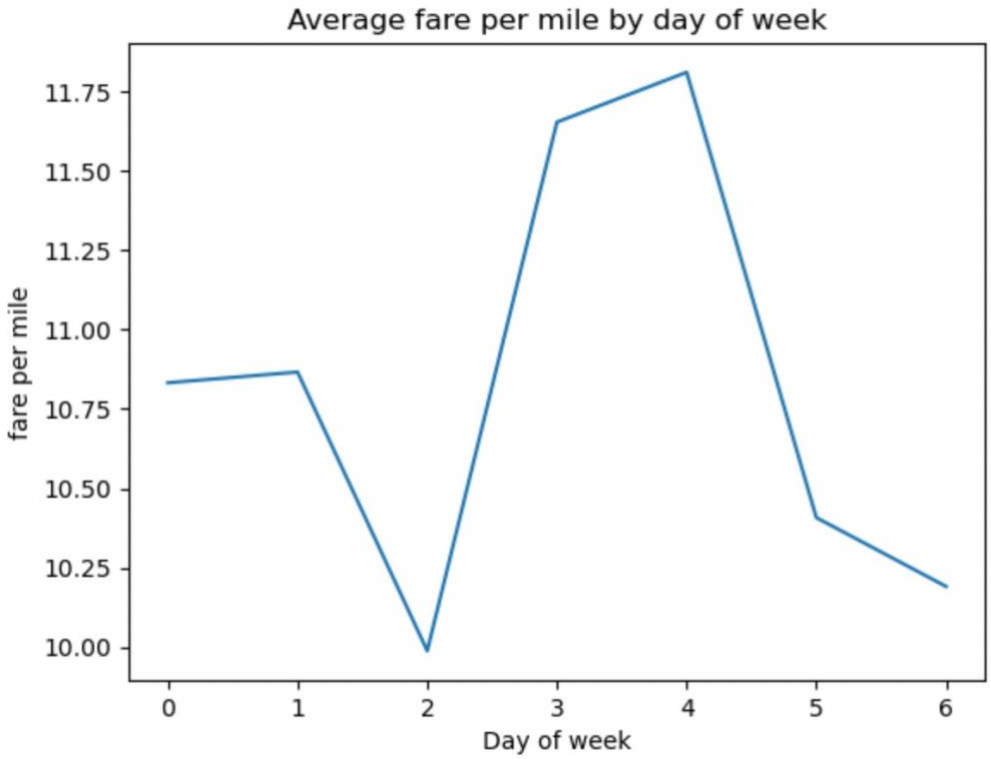
To find out the average fare per mile per passenger for different passenger count, first I created a new dataframe by filtering out all the rows with non zero trip distance and then dividing the fare fare amount of the trip with the trip distance to get the fare per mile for each trip. I then grouped by the dataframe by passenger count to get the average fare per mile per passenger based on number of passenger count. As we can see from below result, fare per mile per passenger increases with decrease in number of passenger in a trip.

```
passenger_count
0.0          inf
1.0    10.421010
2.0     6.347169
3.0     4.117849
4.0     3.427642
5.0     1.713489
6.0     1.344361
dtype: float64
```

3.2.10. Find the average fare per mile by hours of the day and by days of the week

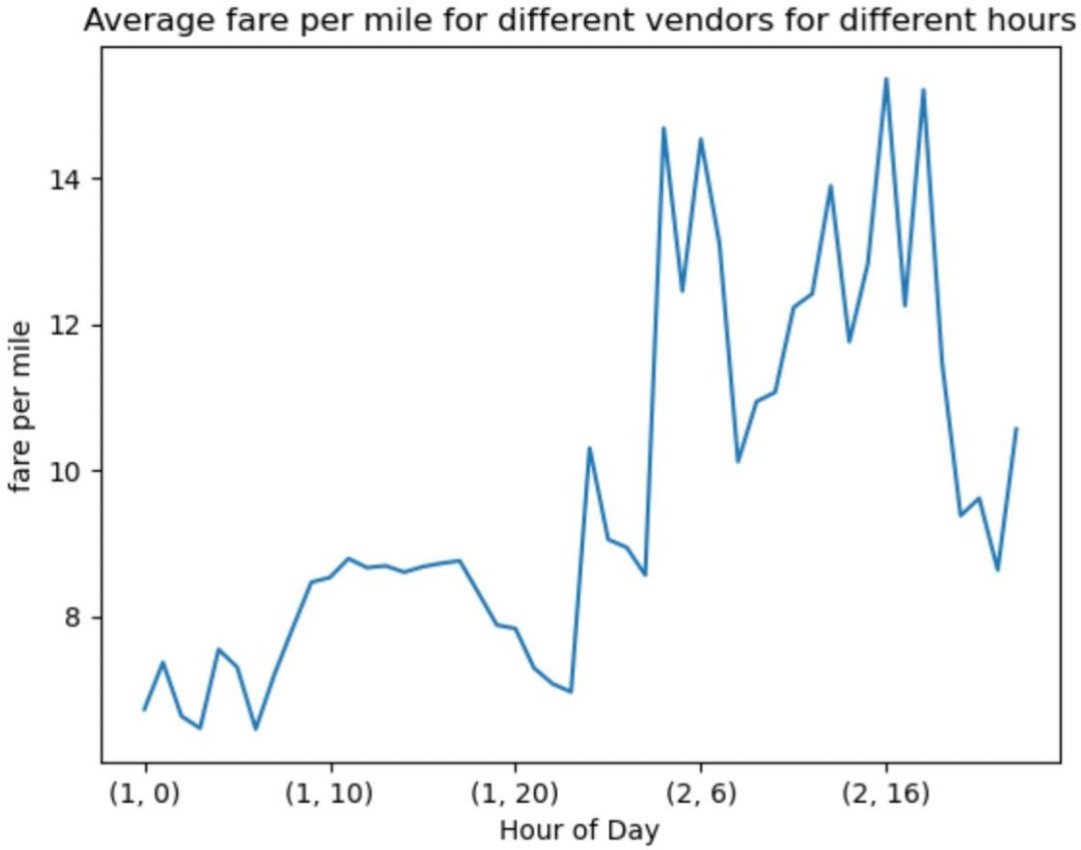
To find the average fare per mile by hours of the day and days of the week, I created two new dataframes from the non zero trips dataframe df_non_zero_trips and then grouped the fare per mile column of dataframe by hours and days of week respectively. Then I plotted the fare per mile against hour/day of the week as shown below.





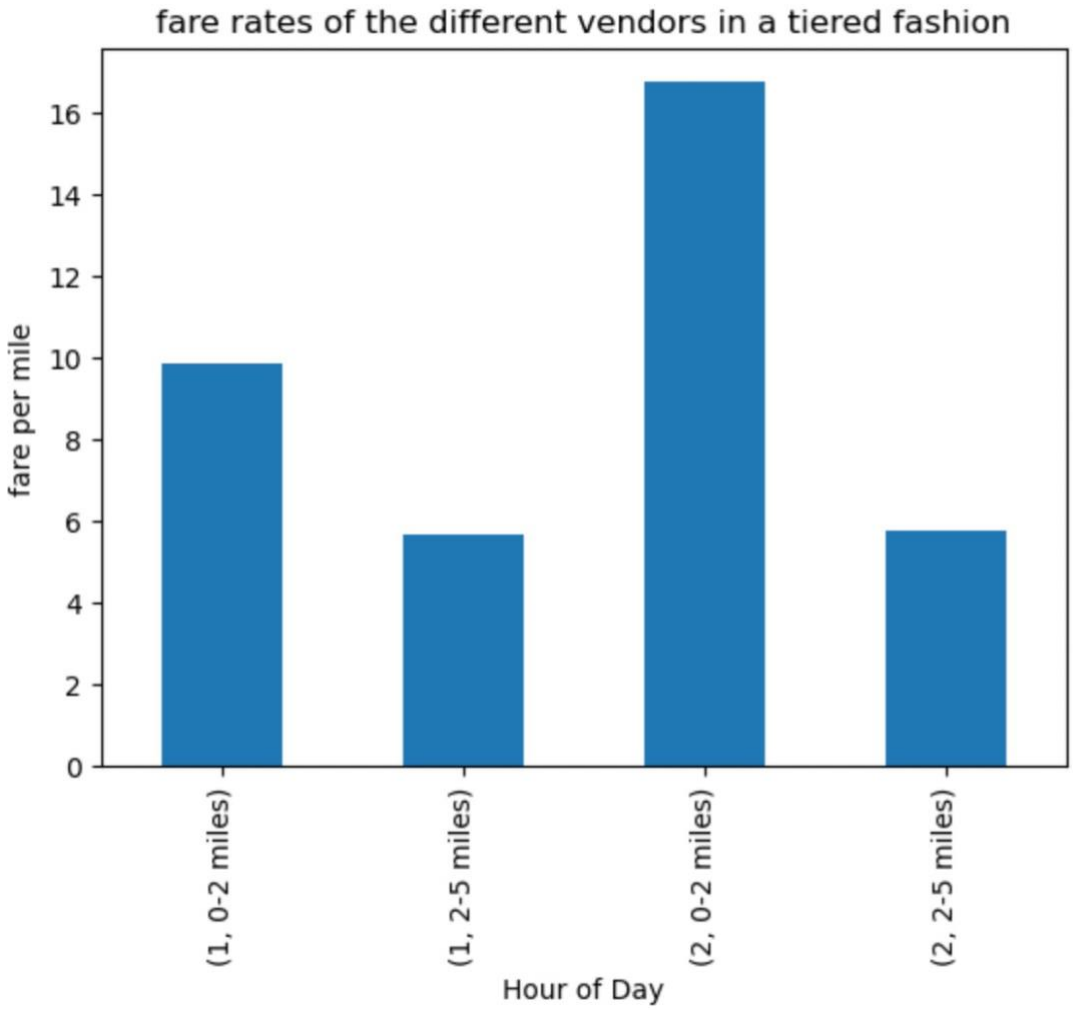
3.2.11. Analyse the average fare per mile for the different vendors

To analyze the average fare per mile for different vendors for each hour of the day, I first grouped the fare per mile column by vendor ID and pickup hour and then calculated the mean to get the average fare per mile for each vendor and each hour of the day. From the graph we can see that, vendor id 1 has lower fare per mile as compared to vendor id 2. We can also observe that vendor 1 fare remains high during the peak time while for vendor 2, the fare varies throughout the day.



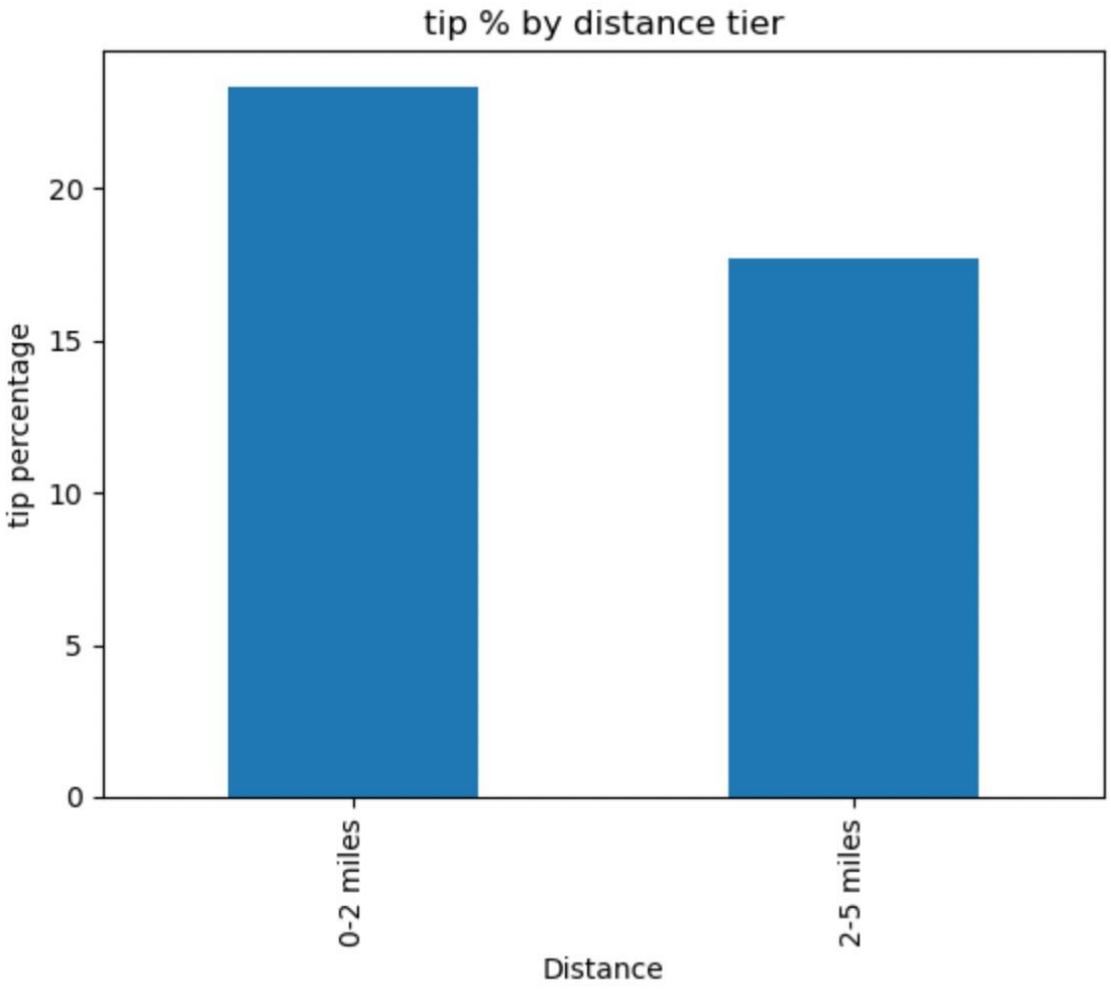
3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

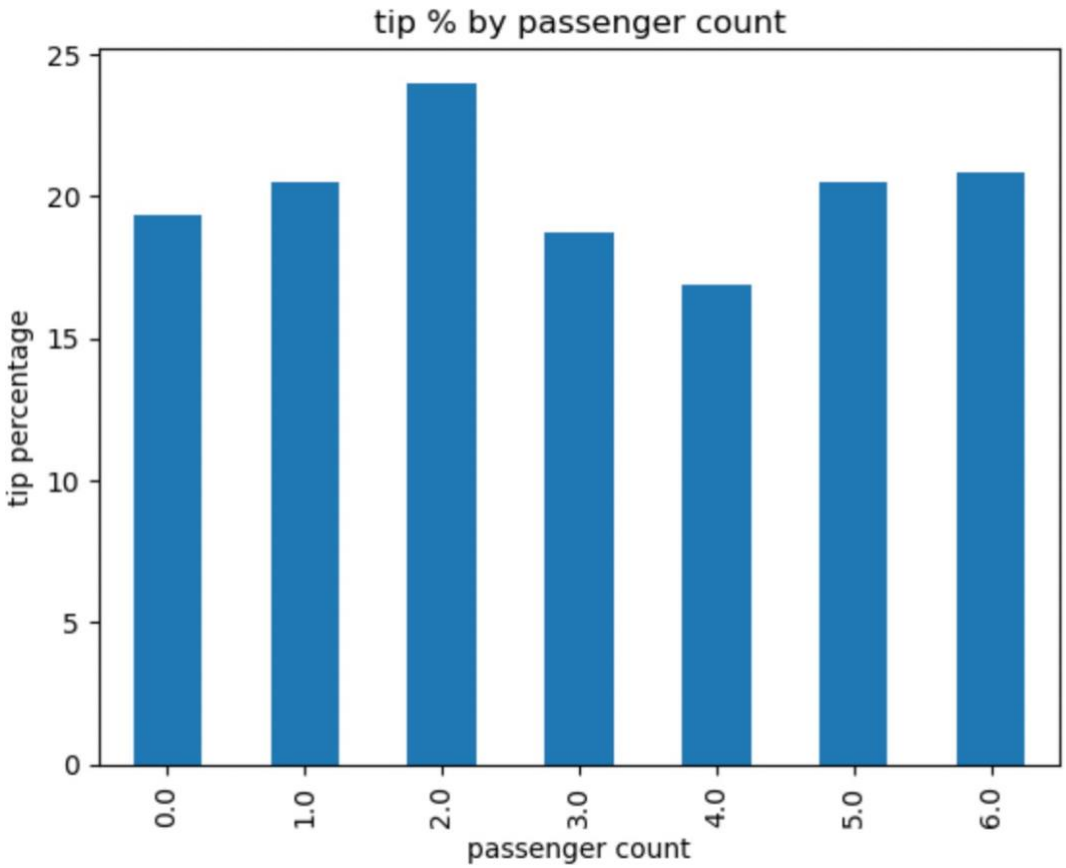
For comparing fare rates of different vendors based on diatnce, I first created a dataframe which divide the data into 3 conditions i.e., less than 2 miles, 2-5 miles and more than 5 miles. Then I created a new dataframe with fare per mile column and grouped it by vendor id and each condition mentioned above and calculated the mean of the fare for each vendor and condition. Finally I plotted the fare per mile against vendor and distance as shown below.

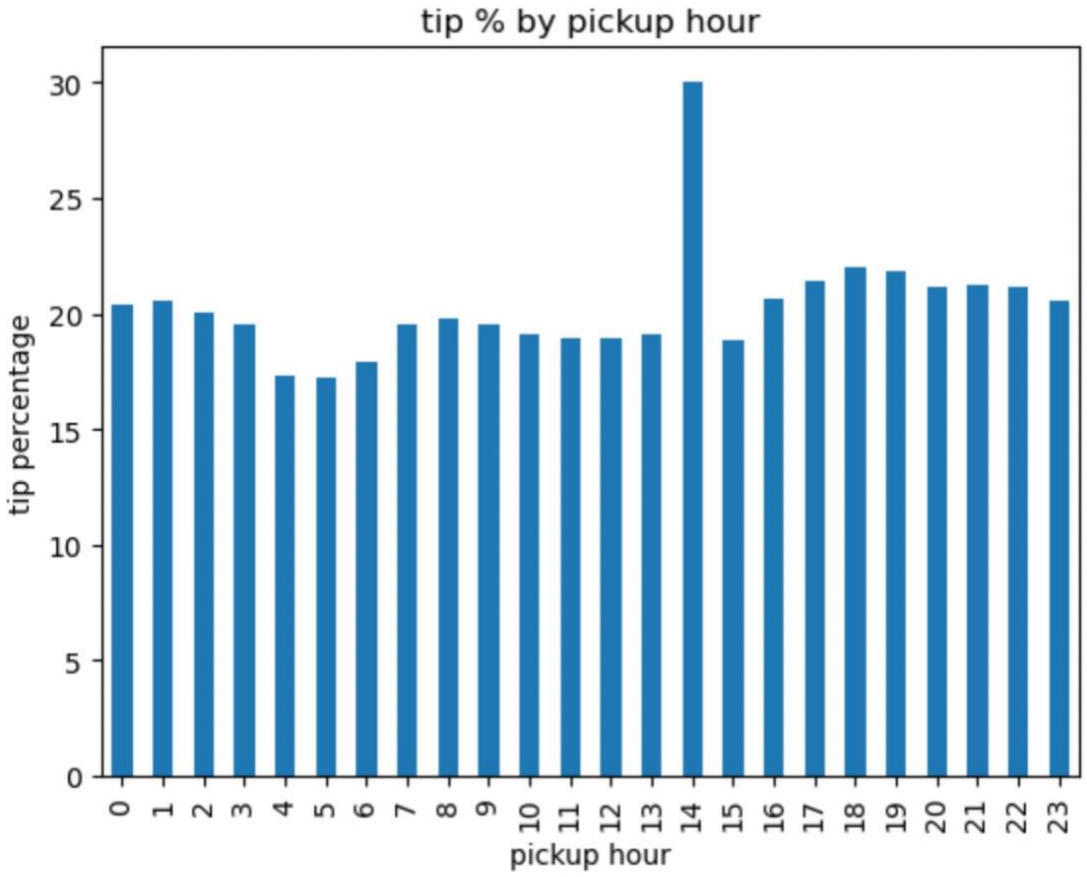


3.2.13. Analyse the tip percentages

To analyze tip percentage on different factors like distance, passenger count, pickup time, first I created 3 different dataframes where I grouped tip percentage column in dataframe by distance_tier, passenger count, pickup hour to get tip_by_distance, tip_by_pass_count, tip_by_pickup_hour dataframes respecrtively. Then, I plotted the tip percent against each factor as shown below. From the below plot, I can conclude that tip percentage is higher for 0-2 miles while it is lower comparatively for 2-5 miles. for passenger count, tip percentage doesn't seem to vary much by passenger count. for pickup hour , I can see that people who travel in afternoon tends to tip more than any other time but overall it remains constant. I can also observe that tip percentage is lower for early morning trips or trips where fare amount if already high as trip distance is high like shown in below plot where tip percentage of trips with 2-5 miles of distance is lower than that of trips with under 2 miles of distance.

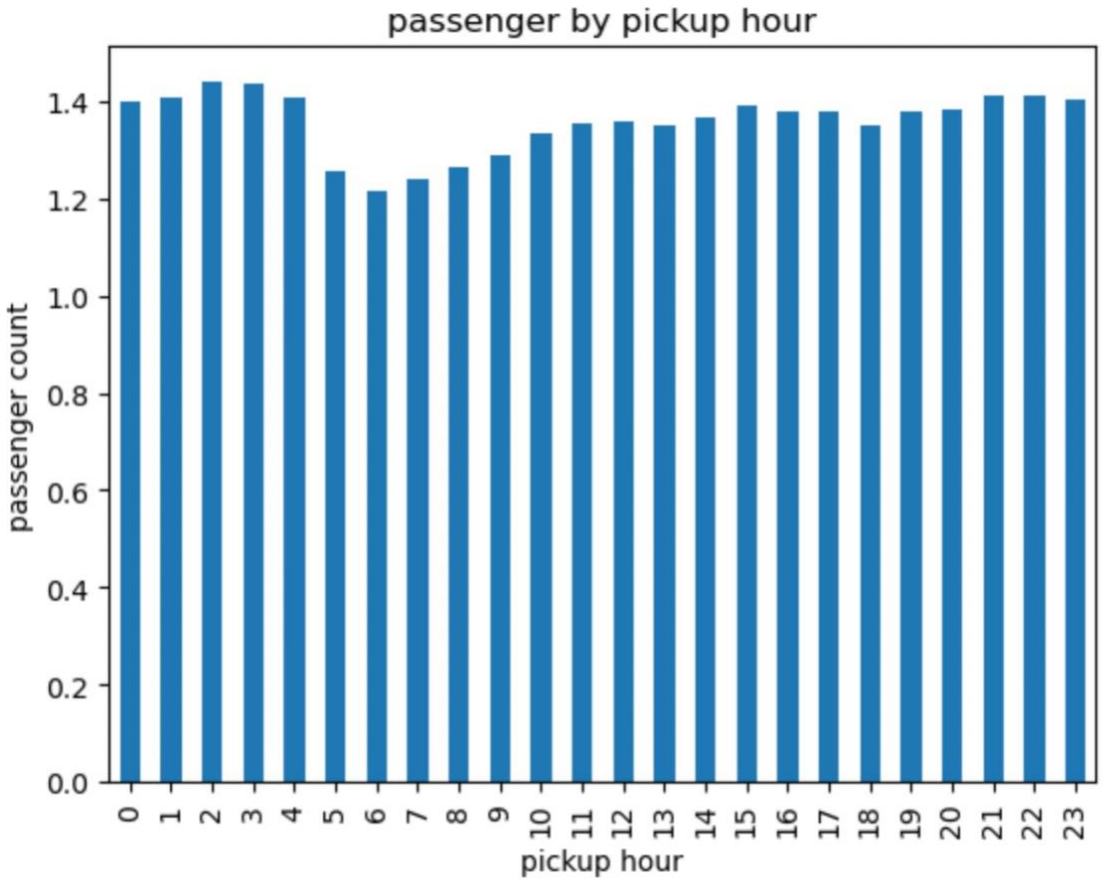

tip % by distance tier
tip percentage
0
5
10
15
20
0-2 miles
2-5 miles
Distance

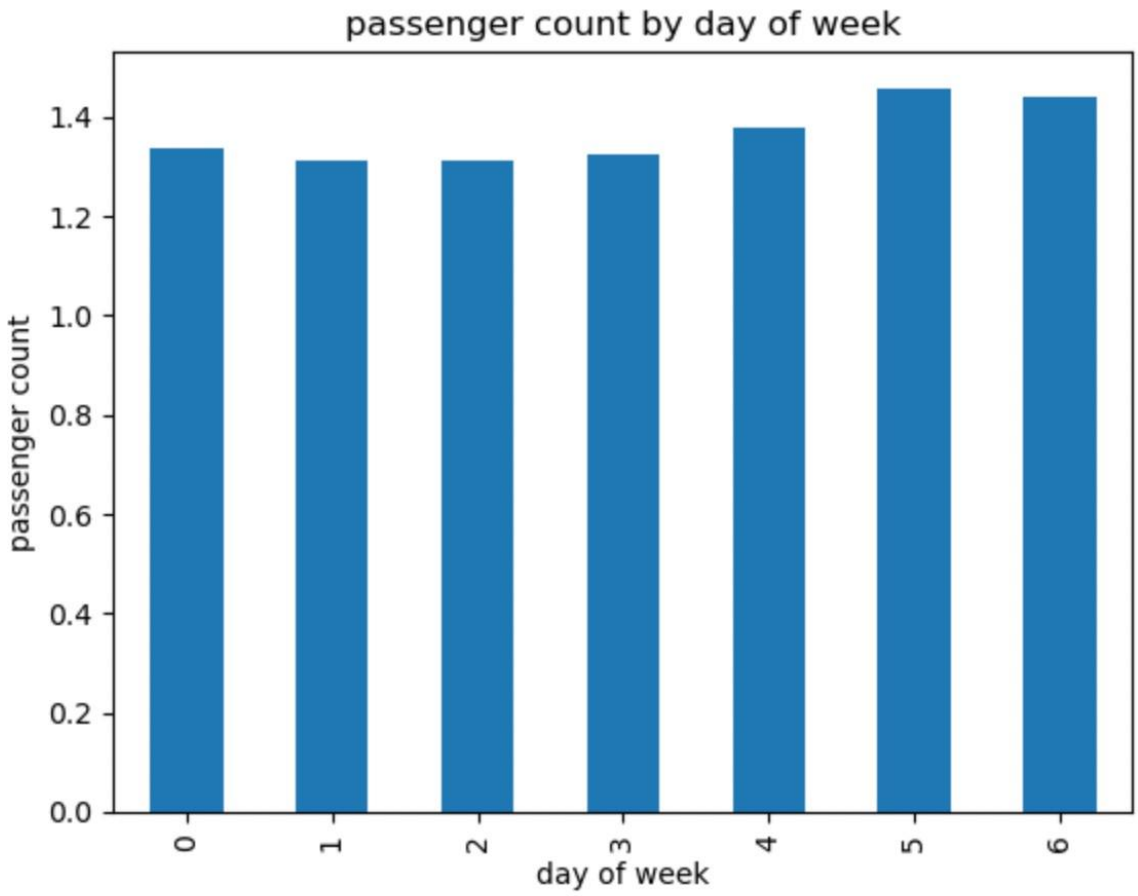

tip % by passenger count
tip percentage
25
20
15
10
5
0
0.0
1.0
2.0
3.0
4.0
5.0
6.0
passenger count



3.2.14. Analyse the trends in passenger count

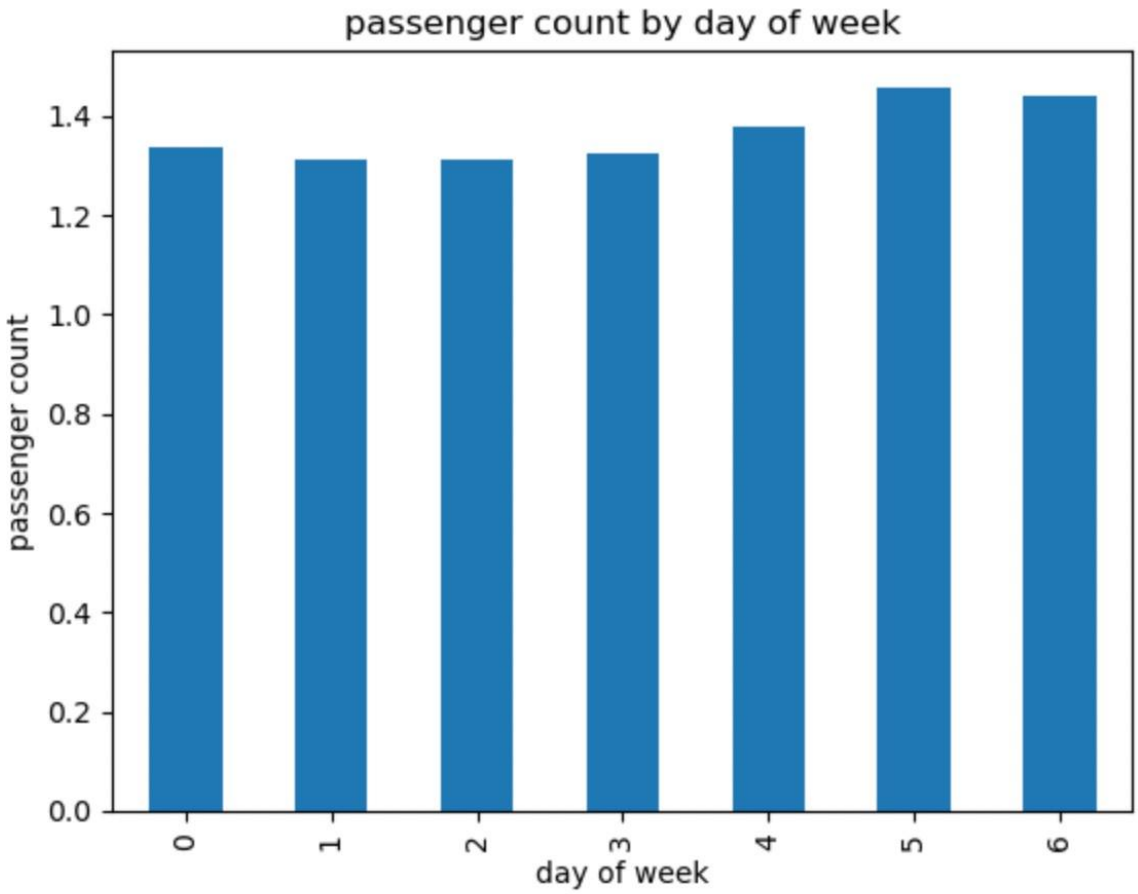
To analyze the variation of passenger count across hours and days of week, first I created two dataframes, first one where I grouped number of passengers by hour of day and second one where grouping is done by days of week and calculated the mean of passenger count for each hour or day of the week. I then plotted the passenger count against hour of the day and day of the week to view the passenger count trend. From my observation, passenger count seems to be lower during early morning and approximately same at other times but little higher for late nights. For days of week also, passenger count approximately remains same but are little higher during weekends. Overall, what we can conclude is that passenger count doesn't really vary based on the hour of the day or day of the week.


passenger by pickup hour
passenger count
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
pickup hour



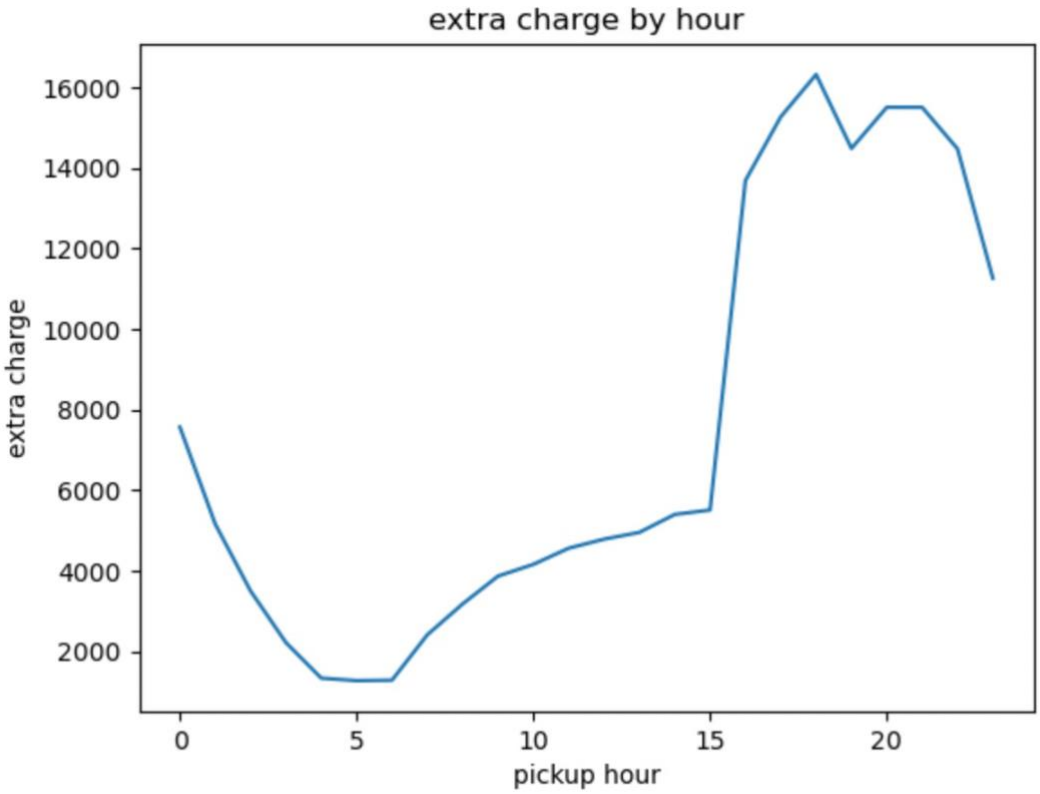
3.2.15. Analyse the variation of passenger counts across zones

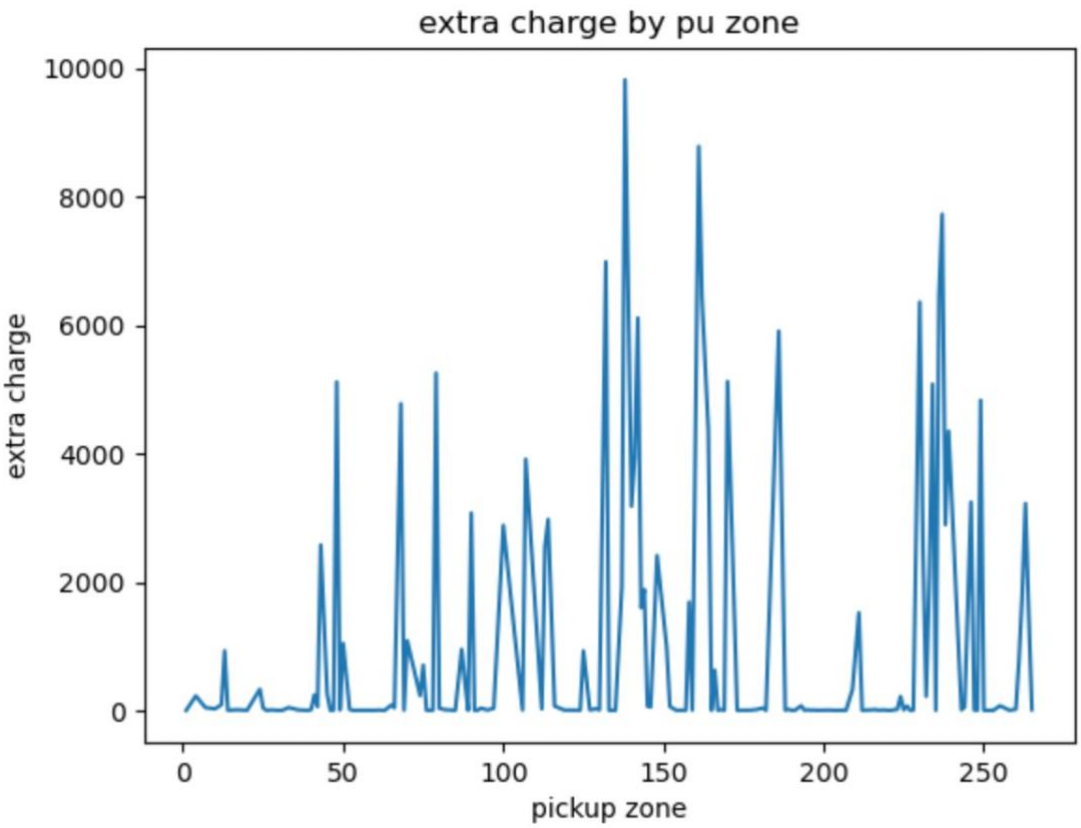
To analyze the variation of passenger counts across zones, I created a new dataframe where I grouped passenger count column on Location id and then calculating mean of passenger count for each zone. Then I plotted the graph between mean passenger count and zones as shown below. What I observed from the plot that passenger count seems to be higher in few zones as compared to other zones. This could be because few zones has more commercial space, residential areas, and entertainment hub as compared to other zones.

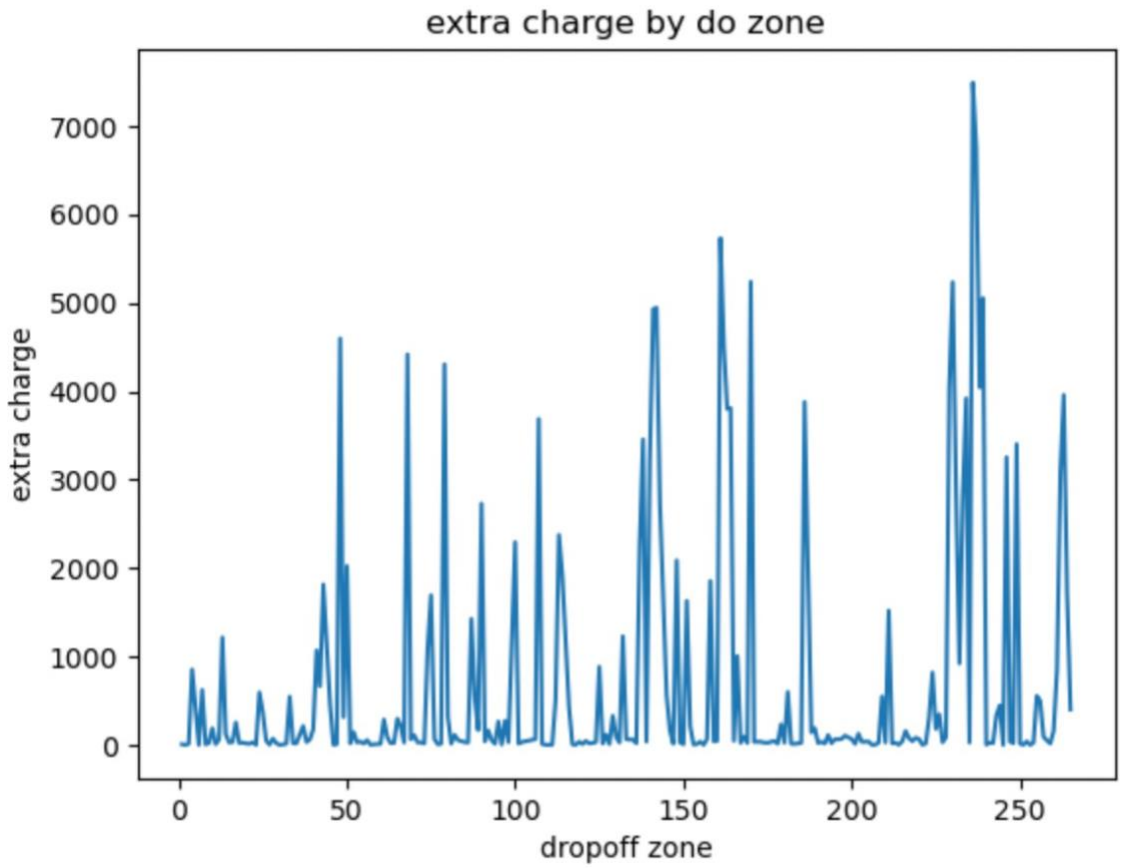


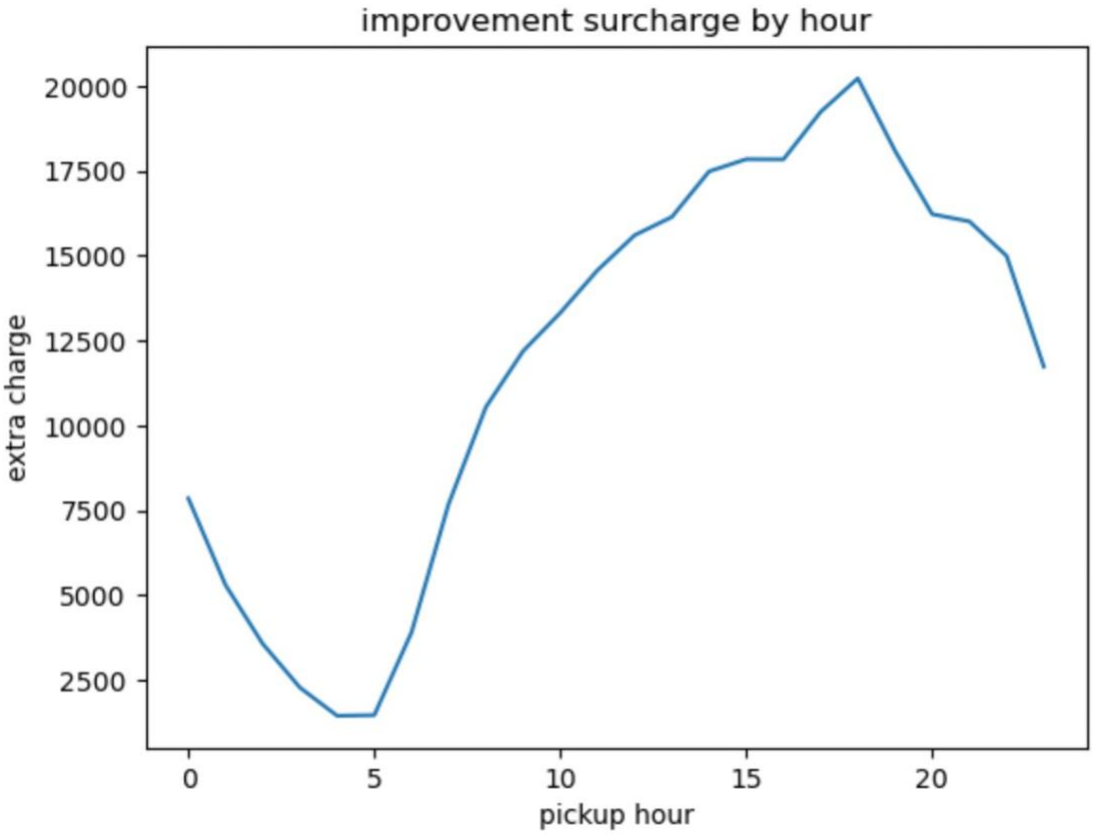
3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

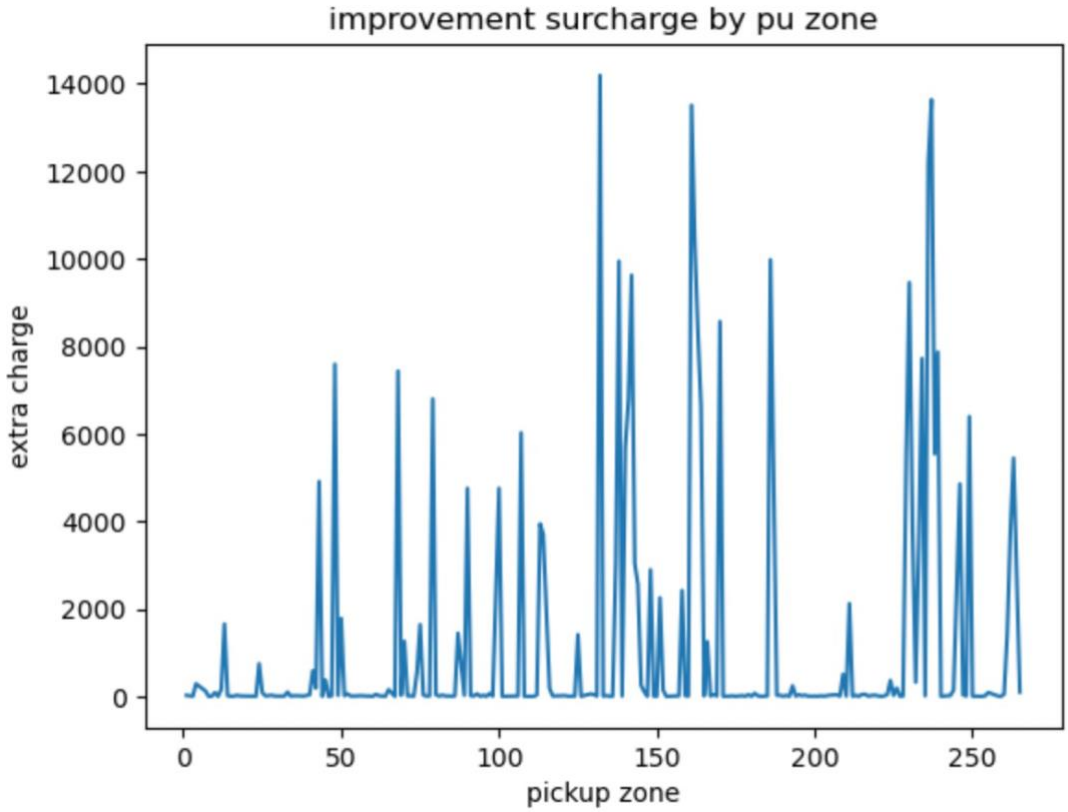
To analyze the pickup/dropoff zones or times when extra charges are applied frequently, I created different dataframes based on pickup location ID, dropoff location id and hour of the day for each extra charge like ‘Extra’, ‘Improvement surcharge’, ‘congestion surcharge’ where amount is greater than 0. Then I created a plot for pickup zone, fropoff zone and hour of the day against each extra charge as shown below. For extra charge, we can see that, it highly depends on what time the trip is happening like during late afternoon and night trips, the extra charge is higher as compared to other time of the day. This could be because the demand and hence the traffic tends to be higher during this time. For pickup and dropoff zone, we can see that ‘extra’ charge is higher for few zones as compared to other zones. We can also see the trend that ‘extra’ charge is more for same zones whether it is pickup or dropoff. This could also be because of the high demand and traffic in few zones because they fall into commercial space or location with high traffic routes. I can see similar trend for other ‘improvement surcharge’ and ‘congestion surcharge’ as well and they both could also be because of similar reasons as for ‘extra’ charge.

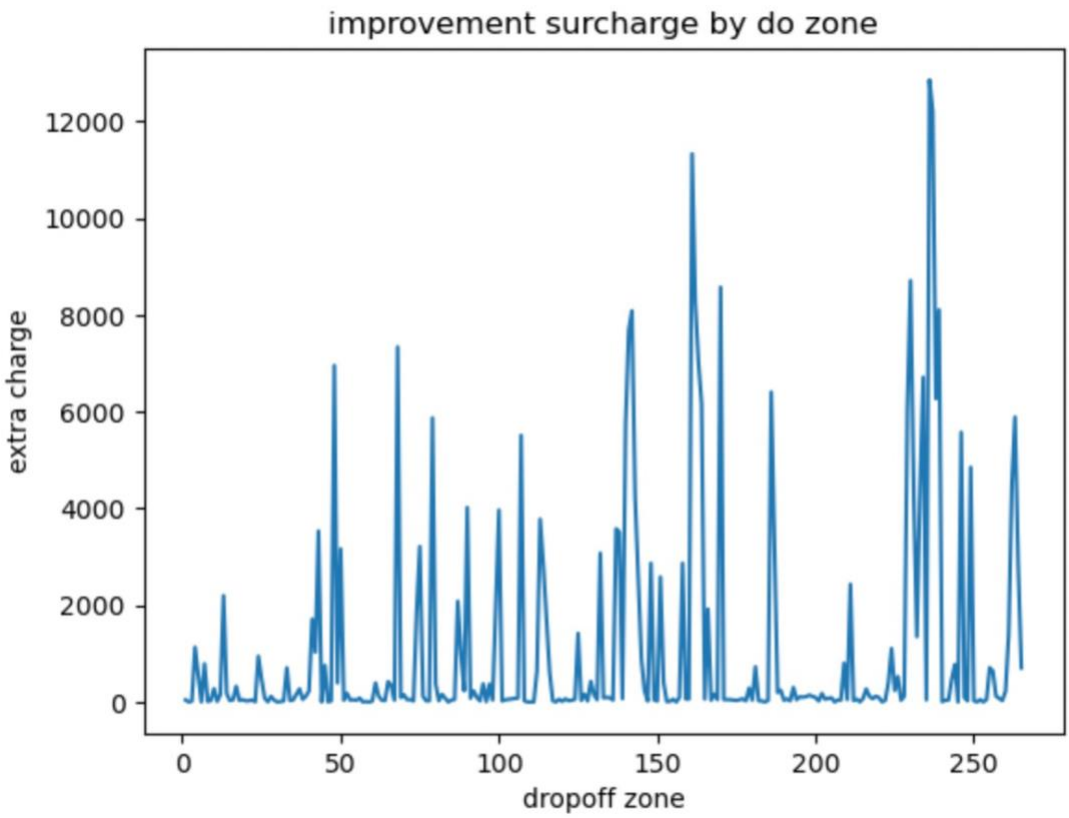

extra charge by hour
16000
14000
12000
10000
8000
6000
4000
2000
extra charge
0
5
10
15
20
pickup hour

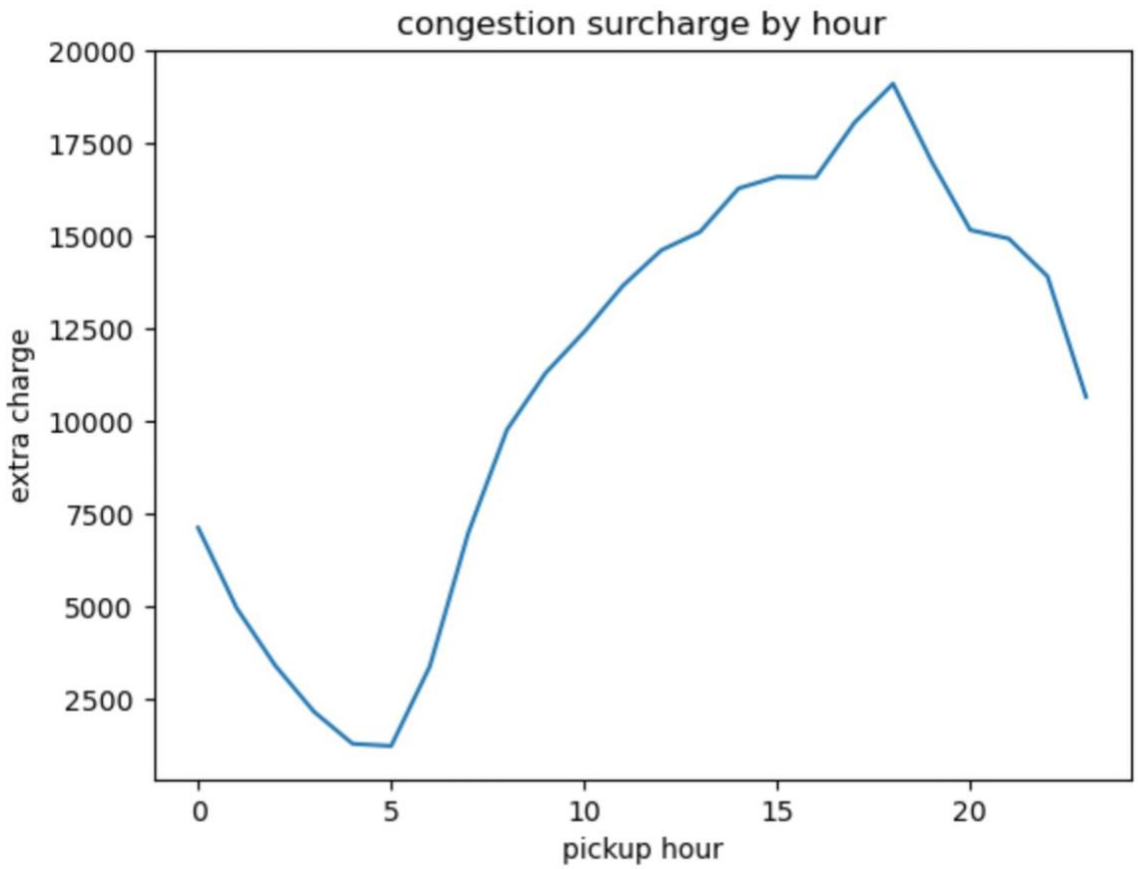

extra charge by pu zone
10000
8000
6000
4000
2000
0
extra charge
0
50
100
150
200
250
pickup zone

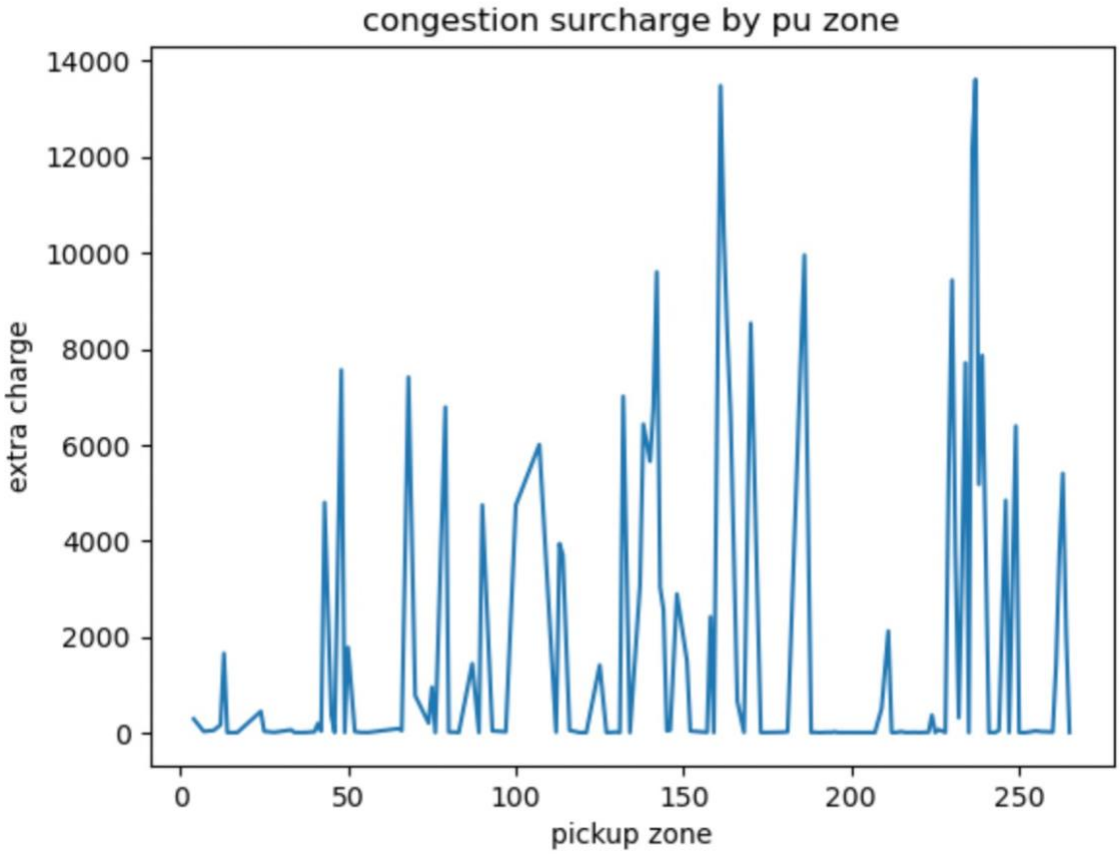

extra charge by do zone
extra charge
7000
6000
5000
4000
3000
2000
1000
0
0
50
100
150
200
250
dropoff zone

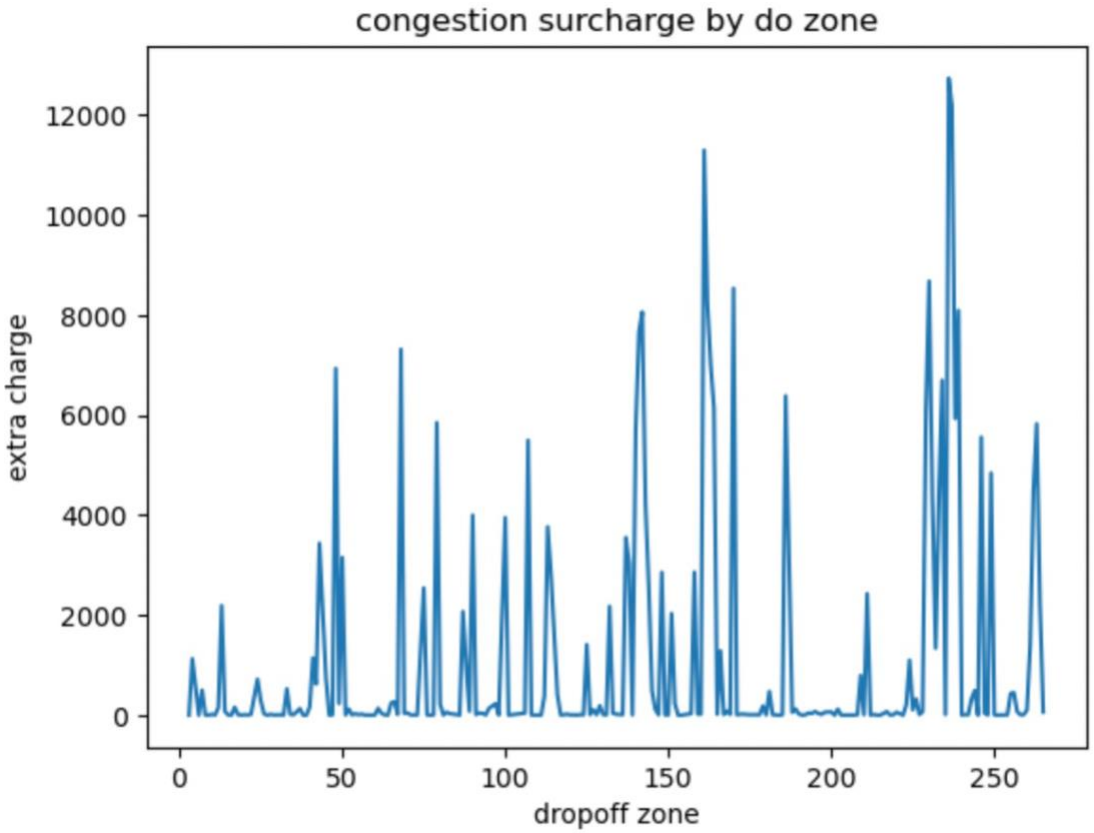

improvement surcharge by hour
extra charge
20000
17500
15000
12500
10000
7500
5000
2500
0
5
10
15
20
pickup hour


improvement surcharge by pu zone
extra charge
14000
12000
10000
8000
6000
4000
2000
0
0
50
100
150
200
250
pickup zone


improvement surcharge by do zone
extra charge
12000
10000
8000
6000
4000
2000
0
0
50
100
150
200
250
dropoff zone


congestion surcharge by hour
20000
17500
15000
12500
10000
7500
5000
2500
extra charge
0
5
10
15
20
pickup hour


congestion surcharge by pu zone
14000
12000
10000
8000
6000
4000
2000
0
extra charge
0
50
100
150
200
250
pickup zone



4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

Based on the data on taxi trips provided and the above analysis done, we can see a clear demand pattern throughout the day
like increase in taxi demand during morning and evening time which could be because of people travelling to work.
We can also see this rising demand during weekdays while on weekends, demand is more in the night time which could be because during weekdays more people travel to and from office while on weekends people are more going to places where they can enjoy during weekends like movies or shopping or other entertainment places.
For recommendations to optimize routing and dispatching based on demand patterns and operational efficiencies,
as we can see from the plot here, the demand of taxis is most during morning and evening time, what we can do is increase availability of more taxis during these hours so that wait time for passengers is reduced and service will also improve.
During weekends, taxis can be positioned near high-demand zones such as commercial areas, entertainment hubs etc.

For optimizing routes, alternative routes can be identified for routes with high congestion. Rerouting options can be made possible like in google maps which tells real time traffic so if there is traffic, drivers can take a different route to reach to their destination.
To conclude, to optimize routing and dispatching, taxi vendors should position more taxis in high demand zones. this high demand varies based on time and days of week, so positioning of taxis in these zones has to be dynamic, otherwise it will lead to more empty trips.
Also, to reduce time between trips, real time traffic can be monitored and accordingly alternative routes can be suggested so that during such time or days when there is high traffic in a specific area, one can chose to reroute for a faster trip.

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

As mentioned in above step, we can see from the analysis that taxis have a high demand in morning and evening because of people going for work on weekdays while on weekends, we can see that the demand is more during night time peobably because of people going for movies or other entertainment hubs. We can also see the demand pattern based on zones, like zones with more offices, commercial areas and entertainment hubs have more demand as compared to other zones. So we can have more taxis available in and near residential areas during morning hours and near areas with offices, we can have more taxis available in evening.
Taxi availability can also be increased in night near entertainment zones and nightlife areas. We can also see that the taxi trips are higher in certain months like May and October while it is low in months of February and August.
This could be because of increase in travel and tourism during summers and autumn season so May and October has more trips while in February, it is colder and it is off season so tourism also decreases.
So, taxi availability can be increased during high demand months also like May, October.

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

Based on the analysis of trip distance, trip duration, demand patterns accross zones, revenue trends, number of trips accross hours and days of week and months of year, we can have few pricing strategies to maximize the revenue.

By positioning more taxis in high demand zones and during high demand hours, we can increase the revenue when demand is high.
We can alternatively have dynamic pricing such that fares are more when demand is high. Also, we can see that fare per mile is
more for shorter trips and less for longer trips, so we can have high base fare and lower incremental fare based on kilometers
such that fare remains balanced but still profitable during shorter trips. For pricing based on hour of day, night surcharges
can be increased so even lesser trips will result into better revenue. Also, it will make more drivers to work during nights and so resulting into more revenue. We can always compare prices among different vendors so that prices are competitive and they are not in loss.